CHƯƠNG 1

Thuật toán và Độ phức tạp tính toán

Học phần này có hai đối tượng nghiên cứu, và tên học phần đã nêu cả hai: *cấu trúc dữ liệu* là cách tổ chức dữ liệu trong bộ nhớ, còn *giải thuật* là dãy các bước xử lý dữ liệu đó. Hai đối tượng này không tách rời nhau. Cùng một bài toán, thay cách tổ chức dữ liệu thì thuật toán phải viết lại; ngược lại, thuật toán quyết định những thao tác nào cần chạy nhanh, và do đó quyết định nên chọn cấu trúc dữ liệu nào.

Chương này xây dựng ba nền tảng cho toàn bộ các chương sau.

1. **Nền tảng về lập trình** (mục 1.1). Tài liệu giả định người đọc chưa từng viết chương trình. Mục này trình bày từ đầu: máy tính thực hiện một chương trình như thế nào, biến là gì, kiểu dữ liệu là gì, ba câu lệnh điều khiển cơ bản, mảng, và hàm. Sinh viên đã học học phần tiên quyết vẫn nên đọc các tiểu mục so sánh ba ngôn ngữ, vì những khác biệt nêu ở đó xuất hiện liên tục về sau.
2. **Nền tảng về khái niệm thuật toán** (mục 1.2). Thuật toán là gì, một dãy bước phải thoả những điều kiện nào mới được gọi là thuật toán, có mấy cách mô tả thuật toán, và làm thế nào để *chứng minh* một thuật toán cho kết quả đúng chứ không chỉ chạy thử vài bộ dữ liệu rồi tin.
3. **Nền tảng về đánh giá hiệu quả** (mục 1.3). Cùng một bài toán thường có nhiều thuật toán. Mục này xây dựng công cụ để so sánh chúng mà không phụ thuộc vào máy tính cụ thể, vào ngôn ngữ lập trình, hay vào trình biên dịch: ký hiệu tiệm cận O, Ω, Θ.

Hai mục cuối chương áp dụng cả ba nền tảng vào các bài toán cụ thể, kết thúc bằng thuật toán phân lớp láng giềng gần nhất — thuật toán học máy đơn giản nhất, và cũng là thuật toán mà toàn bộ chi phí nằm ở một vòng lặp duyệt mảng.

1.1 Chuẩn bị về lập trình

1.1.1 Máy tính thực hiện một chương trình như thế nào

Trước khi viết dòng lệnh đầu tiên, cần một hình ảnh đủ chính xác về những gì xảy ra bên trong máy tính. Hình ảnh này không phải kiến thức bên lề: nó là cơ sở để hiểu vì sao biến hoạt động như vậy, vì sao mảng truy cập nhanh, và vì sao việc đếm số phép toán lại đo được thời gian chạy.

Bộ nhớ. Bộ nhớ chính của máy tính là một dãy rất dài các ô, mỗi ô chứa được một **byte**, tức 8 bit, tức một số nguyên từ 0 đến 255. Các ô được đánh số liên tiếp từ 0; số hiệu của một ô gọi là **địa chỉ** của ô đó. Một máy tính có 8 GB bộ nhớ có khoảng 8×10^9 ô như vậy.

Đặc điểm quan trọng nhất của bộ nhớ chính là **truy cập ngẫu nhiên**: biết địa chỉ thì đọc hay ghi ô đó tốn thời gian như nhau, không phụ thuộc địa chỉ đó nằm ở đâu. Đọc ô số 5 và đọc ô số 5 000 000 tốn thời gian bằng nhau. Tính chất này là lý do mảng cho phép truy cập phần tử thứ i trong thời gian không đổi.

Bộ xử lý. Bộ xử lý lặp lại một chu trình gồm ba việc: lấy lệnh tiếp theo từ bộ nhớ, giải mã xem đó là lệnh gì, rồi thực hiện lệnh đó. Các lệnh mà bộ xử lý hiểu được rất sơ đẳng: cộng hai số, so sánh hai số, chép một giá trị từ ô nhớ này sang ô nhớ khác, nhảy tới một lệnh khác nếu một điều kiện đúng.

Một bộ xử lý phổ thông hiện nay thực hiện được cỡ 10^9 lệnh mỗi giây. Con số này được dùng làm mốc trong suốt tài liệu: khi ta nói một thuật toán cần 10^8 phép toán, nghĩa là nó chạy khoảng một phần mười giây.

Từ mã nguồn tới lệnh máy. Chương trình do người viết bằng ngôn ngữ mà người đọc được, còn bộ xử lý chỉ hiểu lệnh máy. Có ba cách bắc cầu giữa hai thứ đó, và ba ngôn ngữ dùng trong tài liệu đại diện cho đúng ba cách.

Bảng 1.1. Ba cách chuyển mã nguồn thành lệnh máy.

Cách	Cơ chế	Hệ quả
Biên dịch trước	Toàn bộ mã nguồn được dịch thành lệnh máy một lần, trước khi chạy. Kết quả là một tệp chạy được.	Chạy nhanh nhất. Lỗi cú pháp và lỗi kiểu bị phát hiện ngay lúc dịch, trước khi chương trình chạy. Đây là cách của C++.
Thông dịch	Một chương trình khác đọc mã nguồn từng dòng và thực hiện ngay.	Chạy chậm hơn nhiều, nhưng viết và thử nhanh. Lỗi chỉ lộ ra khi dòng lệnh đó thực sự được chạy tới. Đây là cách của Python.
Biên dịch tức thời	Mã nguồn được dịch thành lệnh máy ngay khi hàm được gọi lần đầu, rồi bản đã dịch được giữ lại để dùng cho các lần gọi sau.	Lần gọi đầu chậm, các lần sau nhanh gần bằng biên dịch trước. Đây là cách của Julia.

Nhận xét 1.1. Bảng trên giải thích một hiện tượng sẽ gặp khi làm bài thực hành: cùng một thuật toán, bản C++ có thể chạy nhanh hơn bản Python vài chục lần. Sự chênh lệch đó là chênh lệch *hằng số*, không phải chênh lệch về độ phức tạp. Một thuật toán $O(n \log n)$ viết bằng Python vẫn thắng một thuật toán $O(n^2)$ viết bằng C++ khi n đủ lớn. Toàn bộ tài liệu này quan tâm tới độ phức tạp trước, hằng số sau.

1.1.2 Ba ngôn ngữ dùng trong tài liệu

Mỗi thuật toán trong giáo trình được cài đặt bằng ba ngôn ngữ. Ba bản cài đặt luôn thực hiện đúng cùng một thuật toán và cho cùng kết quả; chúng chỉ khác nhau về cú pháp. Lý do chọn ba ngôn ngữ này như sau.

- **C++** là ngôn ngữ được dùng nhiều nhất trong các kỳ thi lập trình và trên các cổng luyện tập trực tuyến, nên sinh viên bắt buộc phải biết. Nó còn buộc người viết khai báo kiểu và tự quản lý bộ nhớ, nhờ đó những khái niệm như con trỏ, cấp phát động, tràn số nguyên trở nên hữu hình thay vì bị ngôn ngữ che đi.
- **Python** có cú pháp gần với giả mã nhất, nên dùng nó để diễn đạt ý tưởng thuật toán là nhanh và ít gây nhiễu. Đây cũng là ngôn ngữ chính của lĩnh vực trí tuệ nhân tạo, tức lĩnh vực của phần ứng dụng ở cuối mỗi chương.
- **Julia** đánh chỉ số mảng từ 1, trùng với cách đánh số trong giả mã và trong các công thức toán học. Nhờ đó, khi chuyển một công thức trên giấy thành chương trình, ta bớt được một nguồn sai sót rất phổ biến là lệch một đơn vị. Julia cũng cho phép viết ký hiệu toán học trực tiếp trong mã nguồn, chẳng hạn ÷ cho phép chia lấy phần nguyên.

Bảng 1.2 tóm tắt những khác biệt cần nhớ ngay từ đầu. Mỗi dòng của bảng sẽ được giải thích chi tiết ở các tiểu mục sau.

Bảng 1.2. Những khác biệt cơ bản giữa ba ngôn ngữ.

	Julia	Python	C++
Chỉ số mảng đầu tiên	1	0	0
Phân định khối lệnh	từ khoá `end`	thụt đầu dòng	cặp `{ }`
Kết thúc câu lệnh	xuống dòng	xuống dòng	dấu `;`
Khai báo kiểu	không bắt buộc	không bắt buộc	bắt buộc
Chia lấy phần nguyên	`÷`	`//`	`/` với hai số nguyên
Chia lấy phần thực	`/`	`/`	`/` với ít nhất một số thực
Chú thích một dòng	`#`	`#`	`//`
Số nguyên tràn	có, kiểu `Int64`	không, tự mở rộng	có, kiểu `int`

Chọn một ngôn ngữ chính. Sinh viên nên chọn *một* ngôn ngữ làm ngôn ngữ chính và đọc hai ngôn ngữ còn lại như bảng đối chiếu. Học cả ba cùng lúc ở mức thành thạo là không cần thiết và làm loãng thời gian. Nếu mục tiêu là thi lập trình, chọn C++; nếu mục tiêu là làm về trí tuệ nhân tạo, chọn Python.

1.1.3 Chuẩn bị môi trường làm việc

Cần ba thứ: một trình biên dịch hoặc trình thông dịch, một trình soạn thảo, và một cửa sổ dòng lệnh.

Bảng 1.3. Cài đặt và lệnh chạy chương trình.

Ngôn ngữ	Cài đặt	Lệnh chạy tệp `bai.*`
C++	Gói `g++` (Linux: `sudo apt install g++`; Windows: bộ MinGW-w64 hoặc MSYS2)	`g++ -std=c++17 -O2 -o bai bai.cpp` `./bai`
Python	Tải từ `python.org`, bản 3.11 trở lên	`python3 bai.py`
Julia	Tải từ `julialang.org`, bản 1.9 trở lên	`julia bai.jl`

Hai tuỳ chọn trong lệnh biên dịch C++ cần giải thích:

- `-std=c++17` chỉ định phiên bản chuẩn của ngôn ngữ. Không có tuỳ chọn này, một số cú pháp dùng trong tài liệu, chẳng hạn khai báo `auto [a, b] = ...`, sẽ báo lỗi.
- `-O2` yêu cầu trình biên dịch tối ưu mã sinh ra. Không có tuỳ chọn này, chương trình chạy chậm hơn thường 2 đến 5 lần. Mọi số liệu thời gian trong tài liệu đều đo với `-O2`.

Trình soạn thảo nào cũng được, miễn hiển thị đúng tiếng Việt và đánh số dòng. Visual Studio Code là lựa chọn phổ thông vì hỗ trợ cả ba ngôn ngữ.

1.1.4 Chương trình đầu tiên

Chương trình đầu tiên chỉ làm một việc: in một dòng chữ ra màn hình. Mục đích không phải nội dung dòng chữ mà là xác nhận môi trường đã chạy được.

JULIA

```julia
# Đây là chú thích: mọi thứ sau dấu # trên cùng dòng bị bỏ qua.
# println = print line: in ra rồi xuống dòng.
println("Chuong trinh dau tien")
```

PYTHON

```python
# Python cũng dùng dấu # cho chú thích.
# print tự động xuống dòng sau khi in.
print("Chuong trinh dau tien")
```

C++

```cpp
#include <iostream>       // nạp thư viện nhập xuất chuẩn, cần cho cout
using namespace std;      // cho phép viết cout thay vì std::cout

int main() {              // main là hàm chương trình bắt đầu chạy từ đây
    cout << "Chuong trinh dau tien" << endl;   // endl: xuống dòng
    return 0;             // báo hệ điều hành chương trình kết thúc bình thường
}
```

Bản Julia và Python chỉ có một dòng lệnh; bản C++ có sáu dòng. Chênh lệch này không phải do C++ rườm rà một cách vô ích mà do nó đòi hỏi khai báo tường minh mọi thứ. Cụ thể:

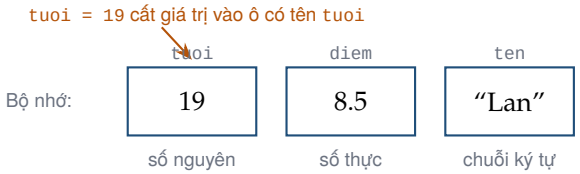
Dòng	Ý nghĩa và lý do có dòng này
1	`#include <iostream>` nạp phần khai báo của thư viện nhập xuất chuẩn. Thiếu dòng này, trình biên dịch không biết `cout` là gì và báo lỗi `'cout' was not declared in this scope`.
2	`using namespace std;` cho phép viết tên ngắn. Không có dòng này, mọi tên trong thư viện chuẩn phải viết đầy đủ: `std::cout`, `std::endl`. Trong dự án lớn người ta tránh dòng này vì có thể gây trùng tên, nhưng với bài tập và bài thi thì nó rút ngắn mã đáng kể.
4	`int main()` khai báo hàm `main`. Đây là nơi chương trình bắt đầu chạy; mọi chương trình C++ phải có đúng một hàm này. Cặp ngoặc nhọn bao toàn bộ thân hàm.
5	Toán tử `<<` đưa dữ liệu ra màn hình và có thể nối liên tiếp nhiều lần: `cout << a << " " << b << endl;`. Ký hiệu `endl` xuống dòng đồng thời đẩy dữ liệu ra thiết bị.
6	`return 0;` báo hệ điều hành rằng chương trình kết thúc bình thường. Giá trị khác 0 quy ước nghĩa là có lỗi.

Dấu chấm phẩy. Mỗi câu lệnh trong C++ phải kết thúc bằng dấu chấm phẩy. Thiếu dấu này là lỗi biên dịch phổ biến nhất của người mới học, và thông báo lỗi thường trỏ vào dòng *tiếp theo* chứ không phải dòng thiếu dấu, vì trình biên dịch chỉ phát hiện ra vấn đề khi đã đọc sang dòng sau.

1.1.5 Biến và kiểu dữ liệu

Định nghĩa 1.1. (*Biến*) **Biến** là một vùng nhớ có tên, dùng để lưu một giá trị. Tên biến dùng để chỉ vùng nhớ đó ở những chỗ khác trong chương trình.

Lệnh **gán** có dạng `tên_biến = giá_trị`: nó tính giá trị ở bên phải rồi ghi vào vùng nhớ mang tên ở bên trái, ghi đè giá trị cũ nếu vùng nhớ đó đã có giá trị.



Hình 1.1. Biến là một ô nhớ có tên. Lệnh gán ghi giá trị vào ô đó, ghi đè giá trị cũ nếu có.

Dấu bằng không phải dấu bằng của toán học. Trong toán học, $x = x + 1$ là một phương trình vô nghiệm. Trong chương trình, `x = x + 1` là một câu lệnh hoàn toàn hợp lệ, đọc là: *lấy giá trị hiện có của x, cộng thêm 1, rồi ghi kết quả trở lại vào x*. Đây là điểm khác biệt gây nhầm lẫn nhiều nhất khi mới học. Một số ngôn ngữ dùng ký hiệu `:=` cho phép gán để tránh nhầm lẫn này; giả mã trong tài liệu dùng mũi tên ← với cùng lý do.

1.1.5.1 Kiểu dữ liệu

Định nghĩa 1.2. (*Kiểu dữ liệu*) **Kiểu dữ liệu** của một biến quy định ba điều: tập giá trị mà biến có thể nhận, số byte bộ nhớ mà biến chiếm, và những phép toán được phép thực hiện trên biến đó.

Bảng 1.4. Các kiểu dữ liệu cơ bản dùng trong tài liệu.

Loại giá trị	Julia	Python	C++
Số nguyên	`Int` (64 bit)	`int` (không giới hạn)	`int` (32 bit), `long long` (64 bit)
Số thực	`Float64`	`float`	`double`
Giá trị lô-gic	`Bool`	`bool`	`bool`
Ký tự	`Char`	không có riêng	`char`
Chuỗi ký tự	`String`	`str`	`string`

JULIA

```julia
tuoi = 19              # Julia tự suy ra kiểu Int từ giá trị 19
diem = 8.5             # dấu chấm thập phân làm giá trị này có kiểu Float64
ten  = "Lan"           # dấu ngoặc kép đôi tạo một String
dat  = true            # giá trị lô-gic, chỉ có true hoặc false

# Nội suy chuỗi: dấu $ chèn giá trị biến vào giữa chuỗi.
println("$ten $tuoi tuoi, diem $diem, dat: $dat")
println(typeof(tuoi), " ", typeof(diem))    # in ra kiểu của biến
```

PYTHON

```python
tuoi = 19              # Python cũng tự suy ra kiểu
diem = 8.5
ten  = "Lan"
dat  = True            # chú ý: True viết hoa chữ T

```

```python
# f-string: chữ f trước dấu ngoặc kép cho phép chèn biến trong cặp { }
print(f"{ten} {tuoi} tuoi, diem {diem}, dat: {dat}")
print(type(tuoi), type(diem))              # in ra kiểu của biến
```

C++

```cpp
int tuoi = 19;          // C++ bắt buộc ghi kiểu trước tên biến
double diem = 8.5;      // double là số thực độ chính xác kép
string ten = "Lan";     // cần #include <string>
bool dat = true;        // true và false viết thường

cout << ten << " " << tuoi << " tuoi, diem " << diem
     << ", dat: " << dat << endl;    // chú ý: bool in ra là 1 hoặc 0
```

Ba khác biệt cần nhớ. Thứ nhất, C++ bắt buộc khai báo kiểu, hai ngôn ngữ kia thì không. Việc khai báo tường minh giúp trình biên dịch bắt được nhiều lỗi ngay lúc dịch, nhưng làm mã dài hơn.
Thứ hai, giá trị lô-gic trong Python viết hoa (`True`, `False`), trong Julia và C++ viết thường (`true`, `false`).
Thứ ba, khi in một biến `bool`, C++ in ra 1 hoặc 0 chứ không in chữ. Muốn in chữ phải dùng `cout << boolalpha`.

1.1.5.2 Vì sao số nguyên có thể tràn

Kiểu `int` của C++ chiếm 32 bit, nên biểu diễn được 2^{32} giá trị khác nhau. Vì phải biểu diễn cả số âm, khoảng giá trị là

$$-2^{31} = -2\,147\,483\,648 \quad \text{đến} \quad 2^{31} - 1 = 2\,147\,483\,647.$$

Khi kết quả một phép tính vượt khỏi khoảng này, hiện tượng **tràn số** xảy ra: các bit vượt quá 32 bị mất, và giá trị thu được sai, thường là một số âm. Chương trình *không báo lỗi*.

C++

```cpp
int a = 2000000000;             // hai tỉ, vẫn nằm trong khoảng của int
int b = a + a;                  // bốn tỉ, VƯỢT khoảng -> tràn
cout << b << endl;              // in ra -294967296, một số âm

long long c = (long long)a + a; // ép sang 64 bit TRƯỚC khi cộng
cout << c << endl;              // in ra 4000000000, đúng
```

Quy tắc thực dụng về tràn số. Trong C++, dùng `long long` cho mọi biến có khả năng vượt khoảng hai tỉ: tổng của một mảng lớn, tích hai số, số phép toán đã đếm. Kiểu `long long` biểu diễn được tới khoảng $9{,}2 \times 10^{18}$.
Phép ép kiểu phải làm *trước* khi tính, không phải sau. Viết `long long c = a + a;` vẫn tràn, vì phép cộng được thực hiện trên hai số `int` rồi kết quả sai mới được gán vào `c`.
Python không có vấn đề này: số nguyên trong Python tự mở rộng theo giá trị. Julia có vấn đề này với kiểu `Int` mặc định là 64 bit; khi cần số lớn hơn phải dùng `BigInt`.

1.1.5.3 Vì sao số thực không chính xác

Số thực được lưu theo chuẩn dấu phẩy động: một số hữu hạn bit cho phần định trị và một số bit cho phần bậc. Vì số bit hữu hạn, chỉ một tập hữu hạn số thực biểu diễn được chính xác, còn lại bị làm tròn. Đặc biệt, các số như $0{,}1$ hay $0{,}2$ không biểu diễn được chính xác trong hệ nhị phân, giống như $1/3$ không biểu diễn được chính xác trong hệ thập phân.

PYTHON

```python
print(0.1 + 0.2)              # in ra 0.30000000000000004, KHÔNG phải 0.3
print(0.1 + 0.2 == 0.3)       # in ra False

# Cách so sánh số thực đúng: kiểm tra sai khác đủ nhỏ
EPS = 1e-9
print(abs((0.1 + 0.2) - 0.3) < EPS)     # in ra True
```

Không so sánh hai số thực bằng dấu bằng. Quy tắc này áp dụng cho cả ba ngôn ngữ. Thay vì viết `x == y`, hãy viết `abs(x - y) < EPS` với `EPS` là một số dương rất nhỏ, thường lấy 10^{-9}. Trong các bài toán hình học, việc quên quy tắc này là nguyên nhân sai kết quả thường gặp nhất.

1.1.6 Nhập dữ liệu

Chương trình thường phải đọc dữ liệu từ bàn phím hoặc từ tệp thay vì gán sẵn trong mã nguồn.

JULIA

```julia
# readline() đọc một dòng và trả về String, kể cả khi dòng đó chứa số.
# parse(Int, s) chuyển String s thành số nguyên; thiếu bước này thì
# phép cộng sẽ nối hai chuỗi thay vì cộng hai số.
n = parse(Int, readline())

# split(s) tách chuỗi theo dấu cách; parse.(Int, ...) có dấu chấm
# nghĩa là áp dụng parse lên TỪNG phần tử của mảng chuỗi.
a = parse.(Int, split(readline()))

println("Đã đọc n = $n và mảng có $(length(a)) phần tử")
```

PYTHON

```python
# input() đọc một dòng và trả về str; int(...) chuyển sang số nguyên.
n = int(input())

# input().split() tách dòng theo dấu cách -> danh sách các chuỗi.
# map(int, ...) áp dụng hàm int lên từng chuỗi.
# list(...) chuyển kết quả của map thành danh sách để dùng được nhiều lần.
a = list(map(int, input().split()))

print(f"Đã đọc n = {n} và mảng có {len(a)} phần tử")
```

C++

```cpp
int n;
cin >> n;                       // toán tử >> đọc và tự chuyển sang kiểu của n

```

```cpp
vector<int> a(n);              // tạo mảng động n phần tử, cần #include <vector>
for (int i = 0; i < n; i++)
    cin >> a[i];               // đọc lần lượt n số vào các ô của mảng

cout << "Da doc n = " << n << " va mang co " << a.size() << " phan tu" << endl;
```

Toán tử >> bỏ qua khoảng trắng. Trong C++, `cin >> x` tự động bỏ qua dấu cách và dấu xuống dòng trước giá trị cần đọc. Nhờ vậy chương trình đọc được dữ liệu bất kể mỗi số nằm trên một dòng riêng hay tất cả nằm trên một dòng. Đây là khác biệt so với Python và Julia, nơi ta phải tự quyết định đọc theo dòng rồi tách.

1.1.7 Các phép toán số học

Bảng 1.5. Các phép toán số học và cú pháp trong ba ngôn ngữ.

Phép toán	Julia	Python	C++
Cộng, trừ, nhân	`+ - *`	`+ - *`	`+ - *`
Chia lấy kết quả thực	`/`	`/`	`/` (số thực)
Chia lấy phần nguyên	`÷`	`//`	`/` (số nguyên)
Chia lấy phần dư	`%`	`%`	`%`
Luỹ thừa	`^`	`**`	`pow()`

Phép chia là chỗ khác biệt lớn nhất và cũng là nguồn lỗi nhiều nhất.

JULIA

```julia
println(7 / 2)      # 3.5   -- dấu / trong Julia LUÔN cho số thực
println(7 ÷ 2)      # 3     -- dấu ÷ lấy phần nguyên (gõ \div rồi Tab)
println(div(7, 2))  # 3     -- cách viết khác của ÷, không cần ký tự đặc biệt
println(7 % 2)      # 1     -- phần dư
println(2^10)       # 1024  -- luỹ thừa
```

PYTHON

```python
print(7 / 2)        # 3.5   -- dấu / trong Python 3 LUÔN cho số thực
print(7 // 2)       # 3     -- hai dấu gạch chéo lấy phần nguyên
print(7 % 2)        # 1     -- phần dư
print(2 ** 10)      # 1024  -- luỹ thừa, chú ý là ** không phải ^
```

C++

```cpp
cout << 7 / 2 << endl;          // 3   -- HAI SỐ NGUYÊN: tự lấy phần nguyên
cout << 7.0 / 2 << endl;        // 3.5 -- có số thực: cho kết quả thực
cout << (double)7 / 2 << endl;  // 3.5 -- ép kiểu để lấy kết quả thực
cout << 7 % 2 << endl;          // 1   -- phần dư
cout << pow(2, 10) << endl;     // 1024 -- cần #include <cmath>
```

Ba cái bẫy của phép chia. Bẫy thứ nhất. Trong C++, `7 / 2` cho 3 chứ không phải 3,5, vì cả hai toán hạng đều là số nguyên. Muốn kết quả thực, phải đảm bảo ít nhất một toán hạng là

số thực.
Bẫy thứ hai. Trong C++, dấu `^` không phải luỹ thừa mà là phép XOR trên từng bit. Viết `2^10` cho kết quả 8, không phải 1024.
Bẫy thứ ba. Phép chia lấy phần nguyên của số âm khác nhau giữa các ngôn ngữ. Python làm tròn *xuống*: `-7 // 2` cho -4. C++ và Julia cắt phần thập phân về phía 0: `-7 / 2` và `-7 ÷ 2` đều cho -3. Khi thuật toán có thể gặp số âm, phải kiểm tra riêng điểm này.

1.1.7.1 Phép chia lấy dư và các ứng dụng

Phép chia lấy dư xuất hiện thường xuyên trong tài liệu, nên đáng dành riêng một đoạn. Với a và m nguyên dương, a mod m là phần dư khi chia a cho m, luôn thuộc $\{0, 1, \ldots, m-1\}$. Ba ứng dụng hay dùng:

- **Kiểm tra chia hết.** a chia hết cho m khi và chỉ khi $a \bmod m = 0$. Chẳng hạn $n \bmod 2 = 0$ nghĩa là n chẵn.
- **Lấy chữ số cuối.** $n \bmod 10$ là chữ số hàng đơn vị của n, còn $n \div 10$ là n sau khi bỏ chữ số đó. Hai phép này kết hợp cho phép tách từng chữ số của một số, kỹ thuật dùng trong sắp xếp cơ số ở chương 2.
- **Quay vòng chỉ số.** Nếu i là chỉ số trong mảng n phần tử thì $(i+1) \bmod n$ là chỉ số tiếp theo, và khi i là phần tử cuối thì kết quả quay về 0. Kỹ thuật này là nền của hàng đợi vòng ở chương 5.

1.1.8 Câu lệnh rẽ nhánh

Câu lệnh rẽ nhánh cho phép chương trình chọn giữa các nhánh xử lý dựa trên một điều kiện.

Bảng 1.6. Các phép so sánh và phép lô-gic.

Ý nghĩa	Julia	Python	C++
Bằng nhau	`==`	`==`	`==`
Khác nhau	`!=`	`!=`	`!=`
Nhỏ hơn, nhỏ hơn hoặc bằng	`< <=`	`< <=`	`< <=`
Và	`&&`	`and`	`&&`
Hoặc	`||`	`or`	`||`
Phủ định	`!`	`not`	`!`

JULIA

```julia
diem = 7.5

if diem >= 8.0
    println("Gioi")
elseif diem >= 6.5          # chú ý: elseif viết liền, không phải else if
    println("Kha")
elseif diem >= 5.0
    println("Trung binh")
else
    println("Khong dat")
end                          # bắt buộc có end để đóng khối lệnh
```

PYTHON

```python
diem = 7.5

if diem >= 8.0:              # dấu hai chấm mở khối lệnh
    print("Gioi")            # thụt vào 4 dấu cách: dòng này thuộc nhánh trên
elif diem >= 6.5:            # elif là viết tắt của else if
    print("Kha")
elif diem >= 5.0:
    print("Trung binh")
else:
    print("Khong dat")
# Không có end: hết thụt đầu dòng là hết khối lệnh.
```

C++

```cpp
double diem = 7.5;

if (diem >= 8.0) {           // điều kiện BẮT BUỘC nằm trong cặp ngoặc đơn
    cout << "Gioi" << endl;
} else if (diem >= 6.5) {    // C++ viết else if tách rời, có dấu cách
    cout << "Kha" << endl;
} else if (diem >= 5.0) {
    cout << "Trung binh" << endl;
} else {
    cout << "Khong dat" << endl;
}
```

Vì sao thứ tự các nhánh quan trọng. Các điều kiện được kiểm tra lần lượt từ trên xuống, và nhánh đầu tiên có điều kiện đúng sẽ được chạy, các nhánh còn lại bị bỏ qua hoàn toàn. Vì vậy nếu đảo thứ tự thành `if diem >= 5.0` trước rồi `elseif diem >= 8.0` sau, thì điểm 9,0 sẽ rơi vào nhánh đầu tiên và bị xếp "Trung binh". Nhánh thứ hai không bao giờ chạy được. Quy tắc: khi các điều kiện lồng nhau theo kiểu bao hàm, luôn đặt điều kiện *chặt nhất* lên trước.

Hai lỗi cú pháp phổ biến. Viết = thay vì ==. Trong C++, `if (x = 5)` là hợp lệ về cú pháp: nó *gán* 5 cho x rồi coi giá trị 5 là điều kiện đúng, nên nhánh luôn chạy. Trình biên dịch thường chỉ đưa ra một cảnh báo, không phải lỗi. Python và Julia thì báo lỗi cú pháp, nên an toàn hơn.
Nhầm and/or với &&/||. Python dùng từ khoá, hai ngôn ngữ kia dùng ký hiệu. Đáng chú ý là Python *cũng* có toán tử `&` và `|`, nhưng chúng là phép toán trên từng bit, không phải phép lô-gic, và cho kết quả khác.

1.1.8.1 Tính chất short-circuit

Cả ba ngôn ngữ đều tính phép và theo kiểu **short-circuit**: nếu vế trái đã sai thì vế phải *không được tính*. Tương tự, với phép hoặc, nếu vế trái đã đúng thì vế phải không được tính.

Tính chất này không chỉ giúp chạy nhanh hơn mà còn được dùng để bảo đảm tính đúng đắn. Đoạn mã sau là ví dụ điển hình, sẽ gặp lại nhiều lần trong tài liệu:

PYTHON

```python
# ĐÚNG: kiểm tra chỉ số hợp lệ TRƯỚC khi truy cập mảng
while j >= 0 and a[j] > x:
    ...

# SAI: khi j = -1, biểu thức a[-1] được tính trước khi kiểm tra j >= 0.
# Trong Python, a[-1] là phần tử CUỐI mảng nên chương trình chạy nhưng
# cho kết quả sai; trong C++ thì truy cập ngoài mảng.
while a[j] > x and j >= 0:
    ...
```

1.1.9 Câu lệnh lặp

Câu lệnh lặp cho phép thực hiện một khối lệnh nhiều lần. Có hai dạng.

1.1.9.1 Vòng lặp với số lần biết trước

JULIA

```julia
# Julia: i chạy lần lượt các giá trị 1, 2, ..., 5
for i in 1:5
    println("Lan thu $i")
end

# Bước nhảy khác 1: cú pháp đầu:bước:cuối
for i in 1:2:9        # 1, 3, 5, 7, 9
    print(i, " ")
end
println()
```

PYTHON

```python
# Python: range(1, 6) sinh ra 1, 2, 3, 4, 5 -- KHÔNG bao gồm số 6.
# Đây là quy ước "cận trên không lấy", nhất quán trong toàn bộ Python.
for i in range(1, 6):
    print(f"Lan thu {i}")

# Ba tham số: range(đầu, cận_trên, bước)
for i in range(1, 10, 2):     # 1, 3, 5, 7, 9
    print(i, end=" ")
print()
```

C++

```cpp
// C++ tách vòng lặp thành ba phần, cách nhau bởi dấu chấm phẩy:
//   khởi tạo ; điều kiện tiếp tục ; bước cập nhật
for (int i = 1; i <= 5; i++) {
    cout << "Lan thu " << i << endl;
}

// i += 2 là cách viết ngắn của i = i + 2
for (int i = 1; i <= 9; i += 2) {     // 1, 3, 5, 7, 9
    cout << i << " ";
}
cout << endl;
```

Ba phần của vòng `for` trong C++.

Dòng	Ý nghĩa và lý do có dòng này
`int i = 1`	Phần khởi tạo, chạy đúng một lần trước khi vào vòng. Biến `i` khai báo ở đây chỉ tồn tại trong vòng lặp.
`i <= 5`	Điều kiện tiếp tục, được kiểm tra *trước* mỗi lần chạy thân vòng. Điều kiện sai thì vòng lặp kết thúc.
`i++`	Bước cập nhật, chạy *sau* mỗi lần chạy thân vòng. `i++` tương đương `i = i + 1`.

Hiểu rõ thứ tự này giải thích được vì sao vòng `for (int i = 1; i <= 5; i++)` chạy đúng 5 lần với `i` nhận các giá trị 1 đến 5, còn `for (int i = 0; i < 5; i++)` cũng chạy 5 lần nhưng `i` nhận 0 đến 4. Dạng thứ hai được dùng khi duyệt mảng đánh chỉ số từ 0.

Cận trên trong ba ngôn ngữ. Ba ngôn ngữ có ba quy ước khác nhau, và đây là nguồn lỗi lệch một đơn vị phổ biến nhất khi chuyển mã giữa chúng:

- Julia `1:n` *có* lấy giá trị n;
- Python `range(0, n)` *không* lấy giá trị n, chỉ tới $n - 1$;
- C++ tuỳ điều kiện: `i <= n` có lấy, `i < n` không.

1.1.9.2 Vòng lặp với số lần chưa biết trước

Khi không biết trước phải lặp bao nhiêu lần, ta dùng vòng `while`: lặp chừng nào điều kiện còn đúng.

Bài toán. Cho số nguyên dương n. Đếm xem phải chia đôi n bao nhiêu lần thì n về 1.

Thuật toán 1.1. *Đếm số lần chia đôi*

1. Khởi tạo biến đếm `so_lan` bằng 0.
2. Trong khi $n > 1$: thay n bằng $n \div 2$ và tăng `so_lan` lên 1.
3. Trả về `so_lan`.

JULIA

```julia
function dem_chia_doi(n)
    so_lan = 0
    while n > 1              # điều kiện được kiểm tra lại trước MỖI vòng
        n = n ÷ 2            # phép chia lấy phần nguyên, n giảm dần
        so_lan = so_lan + 1
    end
    return so_lan
end

println(dem_chia_doi(100))   # 6, vì 100 -> 50 -> 25 -> 12 -> 6 -> 3 -> 1
```

PYTHON

```python
def dem_chia_doi(n):
    so_lan = 0
    while n > 1:
        n = n // 2          # dùng // để n luôn là số nguyên
        so_lan = so_lan + 1
    return so_lan

print(dem_chia_doi(100))    # 6
```

C++

```cpp
int dem_chia_doi(int n) {
    int so_lan = 0;
    while (n > 1) {
        n = n / 2;          // hai số nguyên nên tự lấy phần nguyên
        so_lan = so_lan + 1;
    }
    return so_lan;
}

cout << dem_chia_doi(100) << endl;     // 6
```

Ví dụ 1.1. *Lần vết.* Với $n = 100$, các giá trị n lần lượt là $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$, tức 6 lần chia. Chú ý bước $25 \to 12$: phép chia lấy phần nguyên bỏ phần thập phân, nên $25 \div 2 = 12$ chứ không phải $12{,}5$.

Nhận xét 1.2. Số lần chia đôi để một số n về 1 chính là $\lfloor \log_2 n \rfloor$. Hàm này xuất hiện trong độ phức tạp của rất nhiều thuật toán trong giáo trình: tìm kiếm nhị phân, sắp xếp trộn, các thao tác trên cây cân bằng. Điều đáng ghi nhớ là $\log_2 n$ tăng rất chậm: với $n = 10^6$ nó chỉ khoảng 20, với $n = 10^9$ chỉ khoảng 30.

> **Vòng lặp vô hạn.** Vòng `while` chỉ dừng khi điều kiện trở thành sai. Nếu thân vòng không làm gì để tiến tới điều đó, chương trình lặp vô hạn. Trong ví dụ trên, dòng `n = n ÷ 2` là dòng làm điều kiện tiến tới sai; xoá nó đi thì chương trình treo. Khi viết vòng `while`, hãy tự trả lời câu hỏi: *đại lượng nào giảm đi ở mỗi vòng, và nó giảm tới giới hạn nào thì dừng?*

1.1.10 Mảng

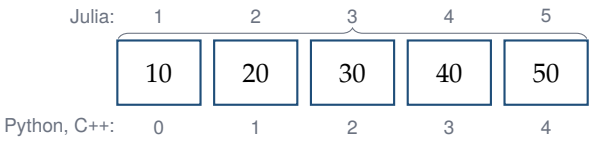
Định nghĩa 1.3. (*Mảng*) **Mảng** là một dãy các phần tử cùng kiểu, nằm liên tiếp trong bộ nhớ, và được truy cập qua **chỉ số**.

Hai đặc điểm trong định nghĩa quyết định mọi tính chất của mảng.

Cùng kiểu. Vì mọi phần tử cùng kiểu nên chúng chiếm số byte bằng nhau. Gọi c là số byte mỗi phần tử và d là địa chỉ của phần tử đầu tiên, địa chỉ của phần tử thứ i tính được bằng công thức

$$\text{địa chỉ}(a_i) = d + i \cdot c.$$

Liên tiếp trong bộ nhớ. Nhờ công thức trên, việc truy cập phần tử thứ i chỉ tốn một phép nhân và một phép cộng, không phụ thuộc i và không phụ thuộc kích thước mảng. Đây là ý nghĩa của phát biểu "truy cập phần tử mảng tốn $O(1)$" sẽ dùng liên tục về sau.



Hình 1.2. Cùng một mảng năm phần tử, hai cách đánh chỉ số. Nhầm lẫn giữa hai cách này sinh ra lỗi lệch một đơn vị, loại lỗi khiến sinh viên mất điểm nhiều nhất trong bài thi thực hành.

JULIA

```julia
a = [10, 20, 30, 40, 50]     # tạo mảng 5 phần tử

println(a[1])                # 10  -- phần tử ĐẦU có chỉ số 1
println(a[5])                # 50  -- phần tử CUỐI có chỉ số length(a)
println(a[end])              # 50  -- từ khoá end cũng chỉ phần tử cuối
println(length(a))           # 5   -- số phần tử

a[3] = 99                    # gán lại phần tử thứ ba
push!(a, 60)                 # thêm 60 vào cuối mảng; dấu ! báo hàm sửa a

for i in 1:length(a)         # duyệt theo chỉ số
    print(a[i], " ")
end
println()

for x in a                   # duyệt theo giá trị, không cần chỉ số
    print(x, " ")
end
println()
```

PYTHON

```python
a = [10, 20, 30, 40, 50]     # trong Python gọi là list

print(a[0])                  # 10  -- phần tử ĐẦU có chỉ số 0
print(a[4])                  # 50  -- phần tử CUỐI có chỉ số len(a) - 1
print(a[-1])                 # 50  -- chỉ số âm đếm ngược từ cuối
print(len(a))                # 5

a[2] = 99                    # gán lại phần tử thứ ba (chỉ số 2)
a.append(60)                 # thêm 60 vào cuối

for i in range(len(a)):      # duyệt theo chỉ số
    print(a[i], end=" ")
print()

for x in a:                  # duyệt theo giá trị
    print(x, end=" ")
print()
```

C++

```cpp
vector<int> a = {10, 20, 30, 40, 50};   // vector: mảng động, cần <vector>

cout << a[0] << endl;        // 10  -- phần tử ĐẦU có chỉ số 0
cout << a[4] << endl;        // 50  -- phần tử CUỐI có chỉ số a.size() - 1
cout << a.size() << endl;    // 5

```

```cpp
a[2] = 99;                  // gán lại phần tử thứ ba
a.push_back(60);            // thêm 60 vào cuối

for (int i = 0; i < (int)a.size(); i++)   // duyệt theo chỉ số
    cout << a[i] << " ";
cout << endl;

for (int x : a)             // duyệt theo giá trị, cú pháp C++11 trở lên
    cout << x << " ";
cout << endl;
```

Bảng 1.7. Bảng đối chiếu các thao tác mảng.

Thao tác	Julia	Python	C++
Tạo mảng từ danh sách giá trị	`a = [1,2,3]`	`a = [1,2,3]`	`vector<int> a = {1,2,3};`
Tạo mảng n số 0	`zeros(Int, n)`	`[0] * n`	`vector<int> a(n, 0);`
Số phần tử	`length(a)`	`len(a)`	`a.size()`
Phần tử đầu	`a[1]`	`a[0]`	`a[0]`
Phần tử cuối	`a[end]`	`a[-1]`	`a.back()`
Thêm vào cuối	`push!(a, x)`	`a.append(x)`	`a.push_back(x);`
Xoá phần tử cuối	`pop!(a)`	`a.pop()`	`a.pop_back();`

Lỗi truy cập ngoài mảng. Truy cập một chỉ số không hợp lệ cho ba hành vi khác nhau, và hành vi của C++ là nguy hiểm nhất:

- **Julia** báo lỗi `BoundsError` và dừng chương trình.
- **Python** báo lỗi `IndexError` và dừng chương trình — trừ khi chỉ số âm, vì chỉ số âm là hợp lệ và đếm ngược từ cuối. Vì vậy `a[-1]` *không* báo lỗi mà trả về phần tử cuối, gây kết quả sai mà không có dấu hiệu gì.
- **C++** với toán tử `[]` *không kiểm tra* chỉ số. Chương trình đọc hoặc ghi vào vùng nhớ không thuộc mảng; kết quả có thể là một giá trị rác, có thể là hỏng dữ liệu khác, có thể là chương trình dừng đột ngột với thông báo `Segmentation fault` ở một chỗ hoàn toàn khác. Khi gỡ lỗi, có thể dùng `a.at(i)` thay cho `a[i]`: hàm này kiểm tra chỉ số và ném ngoại lệ, đổi lại chạy chậm hơn.

1.1.10.1 Mảng hai chiều

Mảng hai chiều dùng để lưu bảng, ma trận, lưới ô vuông. Nó xuất hiện trong quy hoạch động ở chương 4 và trong biểu diễn đồ thị ở chương 6.

PYTHON

```python
# ĐÚNG: mỗi hàng là một danh sách RIÊNG
m = [[0] * 4 for _ in range(3)]     # ma trận 3 hàng, 4 cột, toàn số 0
m[1][2] = 7                          # hàng 1, cột 2

# SAI: [[0] * 4] * 3 tạo 3 THAM CHIẾU tới cùng một danh sách.
# Gán m[1][2] = 7 khi đó làm cả ba hàng cùng đổi.
```

C++

```cpp
// vector lồng vector: 3 hàng, mỗi hàng 4 phần tử, khởi tạo 0
vector<vector<int>> m(3, vector<int>(4, 0));
m[1][2] = 7;

// Cách khác, mảng tĩnh, dùng khi biết trước kích thước tối đa:
int b[3][4] = {};      // cặp ngoặc rỗng khởi tạo toàn bộ về 0
```

JULIA

```julia
m = zeros(Int, 3, 4)    # ma trận 3 hàng 4 cột; Julia có kiểu ma trận riêng
m[2, 3] = 7             # chú ý cú pháp m[hàng, cột], không phải m[hàng][cột]
println(size(m))        # (3, 4)
```

1.1.11 Hàm

Định nghĩa 1.4. (*Hàm*) **Hàm** là một khối lệnh có tên, nhận vào một số **tham số** và trả về một giá trị. Hàm cho phép viết một đoạn xử lý đúng một lần rồi dùng lại ở nhiều chỗ.

Ngoài việc tránh lặp mã, hàm còn có ba lợi ích quan trọng hơn đối với môn học này: nó cho phép *kiểm thử riêng* từng phần của chương trình, nó cho phép *phát biểu và chứng minh* tính đúng đắn của từng phần độc lập, và nó là cơ chế duy nhất để viết thuật toán đệ quy.

JULIA

```julia
# Hàm nhận hai tham số a, b và trả về giá trị lớn hơn.
function lon_hon(a, b)
    if a > b
        return a         # return thoát khỏi hàm ngay lập tức
    else
        return b
    end
end

# Julia còn cho phép viết hàm một dòng theo dạng công thức:
binh_phuong(x) = x * x

println(lon_hon(3, 8))       # 8
println(binh_phuong(5))      # 25
```

PYTHON

```python
def lon_hon(a, b):           # def mở đầu định nghĩa hàm
    if a > b:
        return a
    else:
        return b

def binh_phuong(x):
    return x * x

print(lon_hon(3, 8))         # 8
print(binh_phuong(5))        # 25
```

C++

```cpp
// C++ phải khai báo kiểu của giá trị trả về (int) và của từng tham số.
int lon_hon(int a, int b) {
    if (a > b) return a;
    else       return b;
}

int binh_phuong(int x) {
    return x * x;
}

// Hàm trả về void nghĩa là không trả giá trị nào.
void in_chao(string ten) {
    cout << "Chao " << ten << endl;
}
```

1.1.11.1 Truyền tham số theo giá trị và theo tham chiếu

Câu hỏi cần trả lời: khi hàm sửa tham số của nó, biến gốc bên ngoài có đổi không?

C++

```cpp
// Truyền theo GIÁ TRỊ: hàm nhận một BẢN SAO của x.
void tang_gap_doi(int x) {
    x = x * 2;              // chỉ sửa bản sao, biến gốc không đổi
}

// Truyền theo THAM CHIẾU: dấu & làm x trở thành tên khác của biến gốc.
void tang_gap_doi_ref(int &x) {
    x = x * 2;              // sửa trực tiếp biến gốc
}

int x = 10;
tang_gap_doi(x);            // x vẫn bằng 10
tang_gap_doi_ref(x);        // x bằng 20
```

PYTHON

```python
# Python không có tham chiếu tường minh, nhưng có một quy tắc tương đương:
#   - số nguyên, số thực, chuỗi: KHÔNG sửa được tại chỗ
#   - danh sách, từ điển: SỬA ĐƯỢC tại chỗ
def gan_lai(a):
    a = [9, 9, 9]           # chỉ đổi tên a bên trong hàm, danh sách gốc còn nguyên

def sua_phan_tu(a):
    a[0] = 9                # sửa đúng danh sách gốc

x = [1, 2, 3]
gan_lai(x)
print(x)                    # [1, 2, 3]
sua_phan_tu(x)
print(x)                    # [9, 2, 3]
```

Julia hoạt động giống Python ở điểm này, và bổ sung một quy ước đặt tên: hàm nào sửa tham số đầu vào thì tên kết thúc bằng dấu chấm than. Chẳng hạn `sort(a)` trả về mảng mới và giữ nguyên a, còn `sort!(a)` sắp xếp trực tiếp trên a. Quên dấu chấm than là lỗi khiến chương trình chạy nhưng không có tác dụng gì.

Truyền tham chiếu để tránh sao chép. Trong C++, truyền một `vector` theo giá trị sẽ sao chép toàn bộ mảng. Với mảng một triệu phần tử, mỗi lần gọi hàm tốn thêm $O(n)$ thời gian và $O(n)$ bộ nhớ. Vì vậy quy ước trong tài liệu là: mảng luôn truyền theo tham chiếu. Nếu hàm không cần sửa mảng thì thêm `const` để trình biên dịch kiểm tra giúp:

```
void xu_ly(const vector<int> &a)
```

1.1.11.2 Phạm vi biến và ngăn xếp lời gọi

Biến khai báo bên trong hàm chỉ tồn tại trong lần gọi hàm đó; hết hàm là biến biến mất. Vùng nhớ chứa các biến này gọi là **khung** của lời gọi, và các khung xếp chồng lên nhau tạo thành **ngăn xếp lời gọi**.

Khi hàm A gọi hàm B, khung của B được đặt lên trên khung của A; khi B kết thúc, khung của nó bị bỏ đi và chương trình quay về đúng chỗ trong A. Cơ chế này giải thích vì sao đệ quy hoạt động được, và cũng giải thích vì sao đệ quy quá sâu làm tràn ngăn xếp — nội dung sẽ trình bày ở mục 1.3.6 và chương 3.

Trong Python, một hàm lồng bên trong muốn *gán lại* một biến của hàm bao ngoài thì phải khai báo `nonlocal`; nếu không, phép gán tạo ra một biến địa phương mới và biến bên ngoài không đổi.

PYTHON

```python
def dem_so_le(a):
    dem = 0

    def xu_ly(x):
        nonlocal dem         # BẮT BUỘC: không có dòng này thì dem ngoài không đổi
        if x % 2 == 1:
            dem += 1

    for x in a:
        xu_ly(x)
    return dem
```

1.1.12 Đọc thông báo lỗi

Lỗi là một phần bình thường của việc lập trình. Điều quan trọng là đọc được thông báo lỗi, vì thông báo hầu như luôn chỉ đúng chỗ sai.

Đọc thông báo lỗi đầu tiên, không phải cuối cùng. Với lỗi biên dịch của C++, một lỗi cú pháp thường kéo theo hàng chục thông báo phụ. Sửa lỗi *đầu tiên* rồi biên dịch lại; phần lớn các thông báo còn lại thường tự biến mất. Việc cố sửa lỗi cuối cùng trong danh sách gần như luôn là công cốc.

1.1.13 Khung chương trình nộp bài

Các bài tập của học phần thường có dạng: dòng đầu ghi số bộ dữ liệu kiểm thử, mỗi bộ gồm một vài dòng dữ liệu. Ba khung dưới đây dùng lại được cho hầu hết bài tập.

Bảng 1.8. Các thông báo lỗi thường gặp và nguyên nhân.

Thông báo	Ngôn ngữ	Nguyên nhân thường gặp
`expected ';' before ...`	C++	Thiếu dấu chấm phẩy ở cuối câu lệnh *phía trên* dòng được báo.
`'x' was not declared in this scope`	C++	Dùng biến chưa khai báo, gõ sai tên biến, hoặc thiếu `#include`.
`Segmentation fault`	C++	Truy cập ngoài mảng, dùng con trỏ rỗng, hoặc đệ quy quá sâu. Không kèm số dòng.
`IndentationError`	Python	Thụt đầu dòng không nhất quán, thường do trộn lẫn dấu Tab và dấu cách.
`NameError: name 'x' is not defined`	Python	Dùng biến trước khi gán, hoặc gõ sai tên.
`IndexError: list index out of range`	Python	Chỉ số vượt quá `len(a) - 1`.
`TypeError: ... str ... int`	Python	Quên chuyển chuỗi đọc từ bàn phím sang số bằng `int()`.
`RecursionError`	Python	Đệ quy quá sâu, mặc định giới hạn 1000.
`UndefVarError: x not defined`	Julia	Dùng biến chưa gán.
`MethodError: no method matching`	Julia	Gọi hàm với kiểu tham số không khớp khai báo.
`BoundsError`	Julia	Chỉ số vượt khoảng $1..n$.

C++

```cpp
#include <bits/stdc++.h>     // nạp gần như toàn bộ thư viện chuẩn cùng lúc
using namespace std;

void solve() {               // xử lý MỘT bộ dữ liệu
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    // ... phần xử lý riêng của từng bài ...
}

int main() {
    ios_base::sync_with_stdio(false);   // tắt đồng bộ luồng vào ra với C
    cin.tie(NULL);                      // không đẩy cout ra trước mỗi lần cin

    int t;
    cin >> t;                // số bộ dữ liệu
    while (t--) solve();     // t-- vừa dùng giá trị vừa giảm t đi 1
    return 0;
}
```

Hai dòng tối ưu nhập xuất.

Dòng	Ý nghĩa và lý do có dòng này
14	`ios_base::sync_with_stdio(false)` tắt cơ chế đồng bộ giữa luồng vào ra của C++ và của C. Mặc định hai luồng được đồng bộ để có thể trộn `printf` với `cout`; việc đồng bộ này làm mỗi thao tác chậm đi đáng kể.
15	`cin.tie(NULL)` bỏ ràng buộc "đẩy `cout` ra màn hình trước mỗi lần đọc `cin`". Ràng buộc này chỉ cần khi chương trình hỏi đáp với người dùng, không cần khi đọc dữ liệu từ tệp.

Với dữ liệu cỡ 10^6 số, hai dòng này thường làm tốc độ đọc nhanh lên vài lần và là ranh giới giữa bài nộp đúng hạn và bài nộp quá thời gian. Ngoài ra nên dùng `"\n"` thay cho `endl`, vì `endl` buộc đẩy dữ liệu ra thiết bị sau mỗi lần gọi.

JULIA

```julia
function solve()
    n = parse(Int, readline())
    a = parse.(Int, split(readline()))
    # ... phần xử lý ...
end

function main()
    t = parse(Int, readline())
    for _ in 1:t             # dấu gạch dưới: biến đếm không dùng tới
        solve()
    end
end

main()                       # đặt mọi thứ trong hàm để Julia biên dịch tối ưu
```

PYTHON

```python
import sys
input = sys.stdin.readline   # đọc nhanh hơn hàm input mặc định vài lần

def solve():
    n = int(input())
    a = list(map(int, input().split()))
    # ... phần xử lý ...

def main():
    t = int(input())
    for _ in range(t):
        solve()

main()
```

> **Chú ý.** Tệp tiêu đề `<bits/stdc++.h>` chỉ có trên trình biên dịch `g++` và không thuộc chuẩn C++, nhưng rất tiện khi thi vì khỏi phải nhớ tên từng tệp tiêu đề.
> Trong Julia, việc đặt mọi thứ vào trong hàm không chỉ để gọn: mã lệnh nằm trong hàm được biên dịch tối ưu, còn mã viết ở phạm vi toàn cục chạy chậm hơn nhiều lần.
> Trong Python, `sys.stdin.readline` giữ lại ký tự xuống dòng ở cuối, nên khi đọc chuỗi phải gọi thêm `.strip()`.

1.2 Thuật toán

1.2.1 Khái niệm thuật toán

Từ "thuật toán" (*algorithm*) bắt nguồn từ tên nhà toán học Ba Tư thế kỷ IX Al-Khwarizmi, tác giả một cuốn sách trình bày các quy tắc tính toán với chữ số Ấn Độ. Tên ông khi phiên sang tiếng La-tinh thành *Algoritmi*, và từ đó thành thuật ngữ chỉ một quy trình tính toán có quy tắc rõ ràng.

Định nghĩa 1.5. (*Thuật toán*) **Thuật toán** là một dãy hữu hạn các bước, mỗi bước được mô tả rõ ràng và thực hiện được, nhằm biến *dữ liệu vào* thành *dữ liệu ra* thoả yêu cầu của bài toán.

Định nghĩa này chưa phải một định nghĩa toán học chặt chẽ, vì cụm "mô tả rõ ràng" và "thực hiện được" còn mơ hồ. Việc định nghĩa chặt chẽ khái niệm thuật toán là một thành tựu lớn của toán học nửa đầu thế kỷ XX, với ba mô hình được đề xuất độc lập: máy Turing của Alan Turing, phép tính lambda của Alonzo Church, và các hàm đệ quy của Kurt Gödel. Ba mô hình sau đó được chứng minh là tương đương nhau, tức mô tả cùng một lớp các quy trình tính toán. Giả thiết rằng lớp đó đúng bằng lớp mọi quy trình tính toán mà con người gọi là thuật toán được gọi là **luận đề Church-Turing**. Nội dung này thuộc học phần Lý thuyết tính toán; ở đây chỉ cần ghi nhận rằng khái niệm thuật toán có nền tảng toán học vững chắc, và rằng có những bài toán *không tồn tại* thuật toán giải.

1.2.2 Các đặc trưng của thuật toán

Một dãy các bước chỉ được gọi là thuật toán nếu thoả sáu đặc trưng sau. Với mỗi đặc trưng, ta nêu kèm một phản ví dụ để thấy rõ vì sao nó cần thiết.

1. **Dữ liệu vào** (*input*). Thuật toán nhận không hoặc nhiều đại lượng vào, được xác định rõ về kiểu và về miền giá trị.

Phản ví dụ. Yêu cầu "sắp xếp dãy" mà không nói dãy gồm phần tử kiểu gì thì không lập trình được: sắp xếp số nguyên khác sắp xếp chuỗi ký tự khác sắp xếp bản ghi sinh viên theo điểm.

2. **Dữ liệu ra** (*output*). Thuật toán tạo ra ít nhất một đại lượng ra, có quan hệ xác định với dữ liệu vào.

   Phản ví dụ. Một quy trình chạy xong mà không trả về gì và không thay đổi gì thì không giải quyết bài toán nào.

3. **Tính xác định** (*definiteness*). Mỗi bước phải được mô tả chính xác, không nhập nhằng; hai người khác nhau thực hiện phải cho cùng kết quả.

   Phản ví dụ. Bước "chọn một phần tử tương đối lớn" không xác định. Bước "chọn phần tử ở vị trí giữa đoạn" thì xác định.

4. **Tính hữu hạn** (*finiteness*). Thuật toán phải dừng sau hữu hạn bước với mọi dữ liệu vào hợp lệ.

   Phản ví dụ. Quy trình "kiểm tra tất cả các số nguyên tố" không bao giờ dừng vì có vô hạn số nguyên tố. Quy trình "kiểm tra mọi số nguyên tố nhỏ hơn n" thì hữu hạn.

5. **Tính hiệu quả** (*effectiveness*). Mỗi bước phải đủ sơ đẳng để thực hiện được chính xác trong thời gian hữu hạn.

   Phản ví dụ. Bước "tính π với vô hạn chữ số" không thực hiện được. Bước "tính π với 100 chữ số" thì được.

6. **Tính phổ dụng** (*generality*). Thuật toán phải giải được mọi bài toán trong một lớp bài toán, không chỉ một trường hợp cụ thể.

   Phản ví dụ. Một chương trình in ra kết quả đã tính sẵn cho đúng bộ dữ liệu mẫu của đề bài không phải thuật toán giải bài toán đó.

Nhận xét 1.3. [Tính dừng không phải chuyện hiển nhiên] Trong bốn đặc trưng đầu, tính hữu hạn là đặc trưng khó kiểm tra nhất. Có những dãy bước rất ngắn mà đến nay chưa ai chứng minh được là luôn dừng. Ví dụ nổi tiếng nhất là quy trình Collatz: bắt đầu từ số nguyên dương n, nếu n chẵn thì thay n bằng $n/2$, nếu n lẻ thì thay n bằng $3n + 1$; lặp cho tới khi $n = 1$. Quy trình này đã được kiểm chứng bằng máy tính với mọi n tới cỡ 10^{20}, nhưng chưa ai chứng minh được nó dừng với *mọi* n. Chừng nào chưa có chứng minh, nó chưa được gọi là thuật toán.

1.2.3 Thuật toán, giải thuật và chương trình

Ba từ này hay bị dùng lẫn lộn. Phân biệt như sau.

- **Thuật toán** là ý tưởng và trình tự các bước, độc lập với ngôn ngữ lập trình. Nó có thể viết bằng lời, bằng sơ đồ khối hay bằng giả mã.
- **Giải thuật** trong tài liệu tiếng Việt thường được dùng đồng nghĩa với thuật toán. Khi cần phân biệt, *giải thuật* nhấn mạnh cách giải một bài toán cụ thể, còn *thuật toán* nhấn mạnh tính hình thức của dãy bước. Đề cương học phần dùng cả hai từ theo nghĩa đồng nghĩa, và tài liệu này cũng vậy.
- **Chương trình** là thuật toán đã được viết bằng một ngôn ngữ lập trình cụ thể, kèm mọi chi tiết kỹ thuật mà máy tính đòi hỏi: khai báo kiểu, cấp phát bộ nhớ, xử lý nhập xuất.

Một thuật toán sinh ra nhiều chương trình — trong tài liệu này là ba, ứng với ba ngôn ngữ. Ngược lại, hai chương trình rất khác nhau về mã nguồn vẫn có thể cùng cài đặt một thuật toán. Khi phân tích độ phức tạp, ta phân tích thuật toán chứ không phân tích chương trình.

1.2.4 Các cách mô tả thuật toán

Có bốn cách mô tả một thuật toán, xếp theo mức độ hình thức tăng dần.

1.2.4.1 Cách thứ nhất: mô tả bằng lời

Diễn đạt bằng câu văn thông thường. Ưu điểm là dễ đọc với người chưa biết lập trình; nhược điểm là dễ nhập nhằng và không thể hiện rõ cấu trúc lặp, rẽ nhánh. Cách này chỉ nên dùng để nêu *ý tưởng*, không dùng để đặc tả.

1.2.4.2 Cách thứ hai: liệt kê từng bước

Đánh số các bước và ghi rõ bước nào chuyển sang bước nào. Đây là cách dùng trong các khối "Thuật toán" của tài liệu này. Nó rõ ràng hơn mô tả bằng lời mà vẫn không đòi hỏi biết cú pháp.

1.2.4.3 Cách thứ ba: sơ đồ khối

Biểu diễn bằng hình vẽ với các ký hiệu chuẩn. Ưu điểm là nhìn thấy ngay luồng điều khiển; nhược điểm là cồng kềnh với thuật toán dài, nên chỉ dùng cho thuật toán ngắn hoặc để giảng cho người mới học.

1.2.4.4 Cách thứ tư: giả mã

Giả mã (*pseudocode*) là ngôn ngữ trung gian giữa tiếng Việt và ngôn ngữ lập trình. Nó có cấu trúc điều khiển như ngôn ngữ lập trình nhưng bỏ qua chi tiết cú pháp. Toàn bộ tài liệu này mô tả thuật toán bằng giả mã trước khi cài đặt, và các quy ước dùng như sau.

Bảng 1.9. Quy ước giả mã dùng trong tài liệu.

Ký hiệu	Ý nghĩa
$x \leftarrow v$	Gán giá trị v cho biến x. Dùng mũi tên thay dấu bằng để không nhầm với phép so sánh.
`=`	So sánh bằng.
`div`, `mod`	Chia lấy phần nguyên và chia lấy phần dư.
Thụt đầu dòng	Xác định phạm vi của khối lệnh, giống Python.
`//`	Bắt đầu chú thích tới hết dòng.
`return`	Trả về giá trị và kết thúc thuật toán ngay.
Chỉ số từ 1	Mảng đánh chỉ số từ 1, trùng quy ước toán học và Julia.
Số dòng	Các dòng được đánh số để tiện trích dẫn khi giải thích.

Giả mã không phải mã nguồn rút gọn. Mục đích của giả mã là làm rõ *thuật toán*, nên nó được phép bỏ qua mọi chi tiết không thuộc về thuật toán: khai báo kiểu, cấp phát bộ nhớ, đọc dữ liệu, xử lý trường hợp dữ liệu sai định dạng. Ngược lại, giả mã *không* được phép mơ hồ ở những chỗ thuộc về thuật toán: điều kiện dừng, thứ tự duyệt, giá trị khởi tạo. Một giả mã viết "sắp xếp lại mảng" là giả mã tồi nếu bài toán đang bàn chính là sắp xếp.

1.2.5 Tính đúng đắn của thuật toán

Chạy thử vài bộ dữ liệu rồi thấy kết quả đúng *không* chứng minh được thuật toán đúng. Bộ dữ liệu tiếp theo có thể sai. Mục này trình bày công cụ chuẩn để chứng minh một thuật toán có vòng lặp cho kết quả đúng.

Định nghĩa 1.6. (*Bất biến vòng lặp*) **Bất biến vòng lặp** (*loop invariant*) là một mệnh đề về các biến của chương trình, đúng ngay trước mỗi lần bắt đầu thân vòng lặp.

Để dùng bất biến chứng minh tính đúng đắn, ta phải kiểm tra ba điều.

1. **Khởi tạo.** Mệnh đề đúng trước lần lặp đầu tiên.
2. **Duy trì.** Nếu mệnh đề đúng trước một lần lặp thì nó vẫn đúng trước lần lặp kế tiếp.
3. **Kết thúc.** Khi vòng lặp dừng, mệnh đề cùng với điều kiện dừng cho ta đúng kết luận cần chứng minh.

Hai điều đầu là quy nạp toán học viết lại cho vòng lặp. Điều thứ ba là chỗ ta thu hoạch kết quả.

Ví dụ 1.2. *Chứng minh thuật toán tìm giá trị lớn nhất.* Xét thuật toán quen thuộc sau.

GIẢ MÃ

```
TIM-MAX(a, n)
1  max ← a[1]
2  for i ← 2 to n
3      if a[i] > max
4          max ← a[i]
5  return max
```

Bất biến. Ngay trước mỗi lần chạy thân vòng với giá trị i, biến `max` bằng giá trị lớn nhất trong đoạn $a[1..i-1]$.

Khởi tạo. Trước lần lặp đầu tiên, $i = 2$ nên đoạn cần xét là $a[1..1]$, gồm đúng một phần tử. Dòng 1 đã gán $\texttt{max} = a[1]$, đúng bằng giá trị lớn nhất của đoạn đó.

Duy trì. Giả sử trước lần lặp thứ i, `max` là giá trị lớn nhất của $a[1..i-1]$. Thân vòng so sánh `max` với $a[i]$ và giữ lại giá trị lớn hơn. Vậy sau thân vòng, `max` bằng $\max(a[1..i-1], a[i])$, tức bằng giá trị lớn nhất của $a[1..i]$. Khi biến đếm tăng lên $i+1$, mệnh đề "`max` là lớn nhất của $a[1..(i+1)-1]$" đúng.

Kết thúc. Vòng lặp dừng khi $i = n+1$. Áp bất biến với $i = n+1$: `max` là giá trị lớn nhất của $a[1..n]$, tức của toàn mảng. Đó chính là kết quả cần trả về.

Nhận xét 1.4. Ngoài tính đúng đắn, còn phải chứng minh thuật toán *dừng*. Với vòng lặp `for` có cận cố định thì hiển nhiên. Với vòng `while`, cách chuẩn là chỉ ra một đại lượng nguyên không âm giảm nghiêm ngặt sau mỗi vòng: đại lượng đó không thể giảm mãi nên vòng lặp phải dừng. Đại lượng này gọi là **hàm biến thiên** (*variant*). Trong thuật toán đếm số lần chia đôi ở mục 1.1.9, hàm biến thiên là chính giá trị n.

1.2.6 Ví dụ đầy đủ: thuật toán Euclid

Mục này trình bày trọn vẹn một thuật toán theo đúng trình tự sẽ dùng cho mọi thuật toán về sau: phát biểu bài toán, ý tưởng, thuật toán bằng lời, giả mã, giải thích từng dòng, chứng minh đúng đắn, lần vết, cài đặt, phân tích.

1.2.6.1 Bài toán

> **Bài toán.** Cho hai số nguyên dương a và b. Tìm ước chung lớn nhất của chúng, ký hiệu $\gcd(a, b)$, tức số nguyên dương lớn nhất chia hết cả a lẫn b.

1.2.6.2 Ý tưởng

Cách trực tiếp là thử mọi số từ $\min(a, b)$ giảm dần tới 1 và lấy số đầu tiên chia hết cả hai. Cách này đúng nhưng chậm: với $a = b = 10^9$ nó cần một tỉ phép thử.

Thuật toán Euclid dựa trên một quan sát về ước chung. Nếu d chia hết cả a lẫn b thì d cũng chia hết $a - b$, và tổng quát hơn, chia hết $a \bmod b$. Ngược lại, nếu d chia hết b và $a \bmod b$ thì d chia hết a. Vậy *tập ước chung* của cặp (a, b) trùng với tập ước chung của cặp $(b, a \bmod b)$, và do đó ước chung lớn nhất của hai cặp bằng nhau:

$$\gcd(a, b) = \gcd(b,\ a \bmod b).$$

Phép biến đổi này thay cặp số bằng một cặp số nhỏ hơn hẳn. Lặp lại cho tới khi số thứ hai bằng 0, khi đó $\gcd(a, 0) = a$ và ta có ngay đáp số.

1.2.6.3 Thuật toán

> **Thuật toán 1.2.** *Euclid tìm ước chung lớn nhất*
>
> Dữ liệu vào: hai số nguyên không âm a, b, không đồng thời bằng 0. Dữ liệu ra: $\gcd(a, b)$.
>
> 1. Nếu $b = 0$ thì trả về a và kết thúc.
> 2. Tính $r \leftarrow a \bmod b$.
> 3. Thay $a \leftarrow b$ và $b \leftarrow r$.
> 4. Quay lại bước 1.

1.2.6.4 Giả mã

GIẢ MÃ

```
EUCLID(a, b)
1  while b ≠ 0
2      r ← a mod b        // phần dư của phép chia a cho b
3      a ← b              // số bị chia mới là số chia cũ
4      b ← r              // số chia mới là phần dư
5  return a               // khi b = 0 thì a chính là ước chung lớn nhất
```

> **Giải thích từng bước.**

Dòng	Ý nghĩa và lý do có dòng này
1	Điều kiện lặp. Vòng lặp chạy chừng nào số thứ hai còn khác 0. Khi $b = 0$ thì bài toán đã có lời giải hiển nhiên.
2	Tính phần dư. Đây là bước thu nhỏ bài toán: theo tính chất $\gcd(a, b) = \gcd(b, a \bmod b)$, cặp mới có cùng đáp số nhưng số thứ hai nhỏ hẳn đi.
3–4	Dịch cặp số. Hai dòng này phải giữ đúng thứ tự và phải dùng biến trung gian r ở dòng 2. Nếu viết trực tiếp $b \leftarrow a \bmod b$ rồi $a \leftarrow b$, thì ở dòng thứ hai b đã bị ghi đè và giá trị gán cho a là sai.
5	Trả kết quả. Vòng lặp chỉ thoát khi $b = 0$, và khi đó $\gcd(a, 0) = a$.

1.2.6.5 Tính đúng đắn và tính dừng

Định lý 1.1. (*Tính đúng đắn của thuật toán Euclid*) *Với mọi cặp số nguyên không âm a, b không đồng thời bằng 0, thuật toán Euclid dừng sau hữu hạn bước và trả về* $\gcd(a, b)$.

Chứng minh. **Bất biến.** Trước mỗi lần lặp, $\gcd(a, b)$ bằng ước chung lớn nhất của cặp số ban đầu.

Khởi tạo. Trước lần lặp đầu tiên, cặp (a, b) chính là cặp ban đầu nên mệnh đề hiển nhiên đúng.

Duy trì. Ta chứng minh $\gcd(a, b) = \gcd(b, a \bmod b)$. Đặt $r = a \bmod b$, tức $a = qb + r$ với $q = a \div b$. Nếu d là ước chung của a và b thì $d \mid a - qb = r$, nên d là ước chung của b và r. Ngược lại, nếu d là ước chung của b và r thì $d \mid qb + r = a$, nên d là ước chung của a và b. Hai tập ước chung trùng nhau nên phần tử lớn nhất của chúng bằng nhau. Vậy sau ba dòng 2–4, giá trị $\gcd(a, b)$ không đổi.

Kết thúc. Vòng lặp thoát khi $b = 0$. Khi đó mọi số nguyên dương đều chia hết 0, nên tập ước chung của $(a, 0)$ chính là tập ước của a, và phần tử lớn nhất của nó là a. Kết hợp với bất biến, giá trị a được trả về đúng bằng ước chung lớn nhất của cặp ban đầu.

Tính dừng. Lấy b làm hàm biến thiên. Nó là số nguyên không âm, và sau mỗi vòng lặp giá trị mới của b là $a \bmod b$, luôn nhỏ hơn nghiêm ngặt giá trị cũ của b theo định nghĩa phép chia lấy dư. Một dãy số nguyên không âm giảm nghiêm ngặt không thể vô hạn, nên vòng lặp phải dừng. ■

1.2.6.6 Lần vết

Bảng 1.10. Lần vết thuật toán Euclid với $a = 48$, $b = 18$.

Vòng lặp	a	b	$r = a \bmod b$
1	48	18	$48 = 2 \cdot 18 + 12 \Rightarrow r = 12$
2	18	12	$18 = 1 \cdot 12 + 6 \Rightarrow r = 6$
3	12	6	$12 = 2 \cdot 6 + 0 \Rightarrow r = 0$
—	6	0	thoát vòng lặp, trả về $a = 6$

Kiểm tra: $48 = 2^4 \cdot 3$ và $18 = 2 \cdot 3^2$, nên ước chung lớn nhất là $2 \cdot 3 = 6$. Kết quả khớp.

1.2.6.7 Cài đặt

JULIA

```julia
function euclid(a, b)
    while b != 0
        r = a % b        # phần dư; PHẢI lưu vào biến trung gian trước
        a = b            # số bị chia mới là số chia cũ
        b = r            # số chia mới là phần dư
    end
    return a             # b đã bằng 0 nên a là ước chung lớn nhất
end

# Julia cho phép hoán đổi trên một dòng, ngắn hơn và không cần biến r:
euclid2(a, b) = b == 0 ? a : euclid2(b, a % b)   # bản đệ quy

println(euclid(48, 18))      # 6
println(euclid2(48, 18))     # 6
```

PYTHON

```python
def euclid(a, b):
    while b != 0:
        r = a % b        # phần dư
        a = b
        b = r
    return a

# Python cho phép gán đồng thời, hai vế phải được tính TRƯỚC khi gán,
# nên không cần biến trung gian:
def euclid_ngan(a, b):
    while b != 0:
        a, b = b, a % b
    return a

print(euclid(48, 18))        # 6
print(euclid_ngan(48, 18))   # 6
```

C++

```cpp
int euclid(int a, int b) {
    while (b != 0) {
        int r = a % b;   // C++ không có gán đồng thời nên cần biến r
        a = b;
        b = r;
    }
    return a;
}

// Bản đệ quy, ngắn hơn nhưng tốn ngăn xếp lời gọi:
int euclid_dq(int a, int b) {
    return b == 0 ? a : euclid_dq(b, a % b);
}

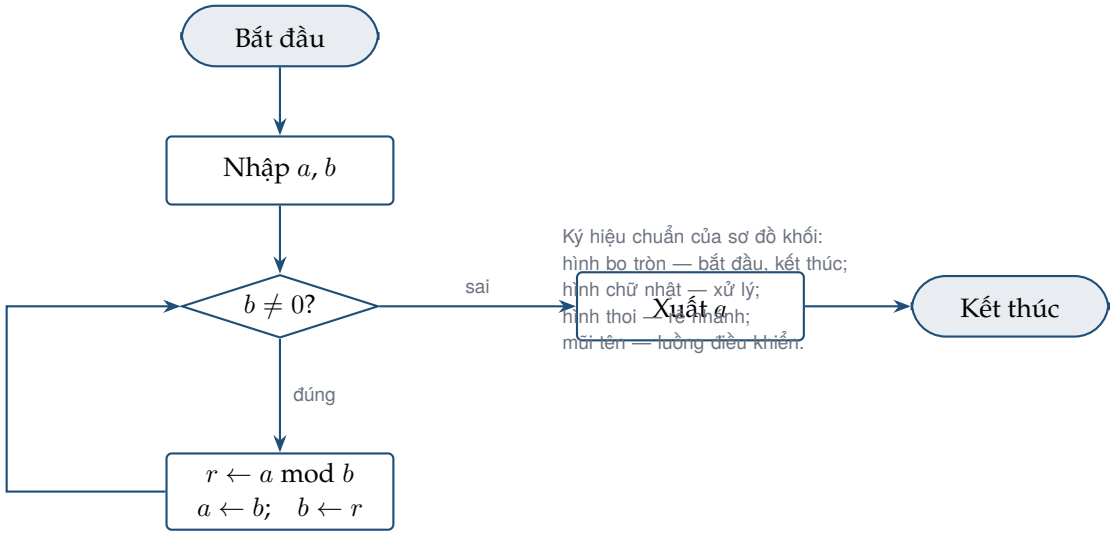
// Thư viện chuẩn C++17 có sẵn: __gcd(a, b) hoặc std::gcd(a, b) trong <numeric>
```

Vì sao Python không cần biến trung gian. Câu lệnh `a, b = b, a % b` tính *cả hai* biểu thức ở vế phải bằng giá trị cũ của a và b, rồi mới gán. Vì vậy nó tương đương với ba dòng dùng biến r. Julia cũng có cú pháp này. C++ thì không, nên bản C++ bắt buộc phải có biến trung

gian — viết `a = b; b = a % b;` sẽ sai vì tới dòng thứ hai thì a đã bị ghi đè.

1.2.6.8 Phân tích

Số vòng lặp của thuật toán Euclid là $O(\log \min(a, b))$. Chứng minh chi tiết dùng dãy Fibonacci và nằm ngoài phạm vi học phần, nhưng có thể thấy trực giác như sau: sau mỗi *hai* vòng lặp, giá trị của a giảm ít nhất một nửa. Với $a, b \le 10^9$, thuật toán chạy nhiều nhất khoảng 60 vòng, so với một tỉ phép thử của cách trực tiếp.



Hình 1.3. Sơ đồ khối thuật toán Euclid tìm ước chung lớn nhất, kèm chú giải các ký hiệu chuẩn.

Bài 1.1. Cài đặt thuật toán Euclid bằng cả ba ngôn ngữ, theo các bước sau.

1. Viết bản dùng vòng lặp, in ra bảng lần vết giống bảng 1.10: mỗi vòng in một dòng gồm a, b, r.
2. Chạy với $(48, 18)$ và đối chiếu với bảng trong tài liệu.
3. Viết thêm bản cài đặt trực tiếp: thử mọi số từ $\min(a, b)$ giảm dần. Đo thời gian của cả hai bản với $a = 999\,999\,937$, $b = 999\,999\,893$ (hai số nguyên tố). Bản Euclid trả lời tức thì, bản trực tiếp mất khoảng một giây.
4. Viết hàm tính bội chung nhỏ nhất theo công thức $\operatorname{lcm}(a, b) = a \cdot b / \gcd(a, b)$. Chú ý trong C++ phải chia trước rồi mới nhân, tức viết `a / __gcd(a,b) * b`, để tránh tràn số ở tích $a \cdot b$.
5. Kiểm thử tự động: sinh 10 000 cặp ngẫu nhiên và so sánh kết quả của hàm mình viết với hàm chuẩn của ngôn ngữ.

1.3 Độ phức tạp tính toán

1.3.1 Vì sao cần đánh giá độ phức tạp

Cùng một bài toán thường có nhiều thuật toán. Câu hỏi "thuật toán nào tốt hơn" cần một câu trả lời không phụ thuộc vào những yếu tố ngẫu nhiên. Ba cách trả lời sau đều không dùng được.

- **Đo thời gian chạy thực tế.** Kết quả phụ thuộc máy tính, hệ điều hành, trình biên dịch, các chương trình khác đang chạy, và bộ dữ liệu thử. Đo lại trên máy khác cho con số khác.
- **Đếm số dòng lệnh trong mã nguồn.** Một dòng có thể chứa một phép gán, cũng có thể chứa một lời gọi hàm thư viện chạy hàng triệu phép toán.
- **Đếm tổng số phép toán thực hiện.** Con số này đúng nhưng phụ thuộc bộ dữ liệu cụ thể và thường là một đa thức cồng kềnh, khó so sánh giữa hai thuật toán.

Cách được dùng là: đếm số phép toán như một *hàm của kích thước dữ liệu vào*, rồi bỏ qua các hằng số và các số hạng bậc thấp, chỉ giữ lại *tốc độ tăng* của hàm đó. Kết quả gọi là độ phức tạp tiệm cận.

Nhận xét 1.5. [Vì sao chỉ quan tâm tốc độ tăng] Hằng số phụ thuộc máy tính và ngôn ngữ; một máy nhanh gấp mười lần chỉ chia hằng số cho mười. Ngược lại, tốc độ tăng là thuộc tính của bản thân thuật toán. Bảng 1.13 cho thấy một máy nhanh gấp mười lần không cứu nổi một thuật toán $O(n^2)$ khi n tăng gấp mười: thời gian chạy vẫn tăng gấp mười.

1.3.2 Mô hình máy tính và phép toán cơ bản

Để việc đếm có nghĩa, cần thống nhất "một phép toán" là gì. Tài liệu dùng mô hình **máy truy cập ngẫu nhiên**, viết tắt là mô hình RAM, với các quy ước sau.

- Các thao tác sau tốn thời gian như nhau và được tính là *một* phép toán: một phép cộng, trừ, nhân, chia, so sánh hai số; một phép gán; một lần đọc hoặc ghi một ô mảng khi đã biết chỉ số; một lệnh nhảy.
- Bộ nhớ vô hạn và mỗi ô chứa được một số nguyên bất kỳ.
- Chương trình chạy tuần tự, không song song.

Mô hình này đơn giản hoá thực tế ở vài chỗ — phép nhân thực ra chậm hơn phép cộng, và bộ nhớ đệm làm việc truy cập ô nhớ gần nhau nhanh hơn ô nhớ xa nhau — nhưng những khác biệt đó chỉ ảnh hưởng tới hằng số, không ảnh hưởng tới tốc độ tăng.

Định nghĩa 1.7. (*Phép toán tích cực*) **Phép toán tích cực** của một thuật toán là phép toán có số lần thực hiện ít nhất cũng tỉ lệ thuận với tổng số phép toán của thuật toán.

Nhờ khái niệm này, thay vì đếm mọi phép toán ta chỉ cần đếm một loại. Với thuật toán sắp xếp, phép toán tích cực thường là phép so sánh hai phần tử; với thuật toán tìm kiếm là phép so sánh khoá; với thuật toán trên đồ thị là số lần duyệt qua một cạnh.

1.3.3 Cách đếm số phép toán

Ký hiệu $T(n)$ là số phép toán tích cực khi kích thước dữ liệu vào là n. Ba đoạn chương trình sau minh hoạ ba trường hợp cơ bản.

C++

```cpp
// Đoạn 1: không có vòng lặp
int x = a[0];                          // T(n) = 1

// Đoạn 2: một vòng lặp chạy n lần
long long tong = 0;
for (int i = 0; i < n; i++)
```

```
    tong += a[i];                    // thân vòng chạy n lần -> T(n) = n

// Đoạn 3: hai vòng lặp lồng nhau, mỗi vòng n lần
int dem = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        dem++;                       // thân vòng chạy n * n lần -> T(n) = n²
```

Nguyên tắc đếm. Số lần thực hiện một câu lệnh bằng tích số lần lặp của tất cả các vòng lặp bao quanh nó. Vì vậy khi phân tích, ta tìm câu lệnh nằm *sâu nhất* trong các vòng lặp và đếm số lần nó chạy; các câu lệnh ở ngoài đóng góp ít hơn nên không ảnh hưởng tới bậc của $T(n)$.

1.3.4 Ký hiệu tiệm cận

Định nghĩa 1.8. (*Ký hiệu O*) Cho hai hàm $f, g : \mathbb{N} \to \mathbb{R}^+$. Ta viết $f(n) = O(g(n))$ nếu tồn tại hằng số $c > 0$ và số tự nhiên n_0 sao cho

$$f(n) \le c \cdot g(n) \quad \text{với mọi } n \ge n_0.$$

Đọc là "f có bậc không vượt quá g". Ký hiệu O cho *cận trên*: nó nói thuật toán không chậm hơn g, nhưng có thể nhanh hơn.

Hai chi tiết trong định nghĩa cần chú ý. Thứ nhất, bất đẳng thức chỉ cần đúng *từ một điểm trở đi*, nên hành vi của hàm với n nhỏ hoàn toàn không quan trọng. Thứ hai, hằng số c được phép lớn tuỳ ý, nên mọi hằng số nhân đều bị bỏ qua.

Ví dụ 1.3. *Chứng minh trực tiếp từ định nghĩa.* Chứng minh $3n^2 + 5n + 100 = O(n^2)$.

Ta cần tìm c và n_0 cụ thể. Với mọi $n \ge 10$ ta có $5n \le n^2$ (vì $5 \le n$) và $100 \le n^2$ (vì $n^2 \ge 100$). Do đó

$$3n^2 + 5n + 100 \le 3n^2 + n^2 + n^2 = 5n^2 \quad \text{với mọi } n \ge 10.$$

Vậy định nghĩa được thoả với $c = 5$ và $n_0 = 10$.

Định nghĩa 1.9. (*Ký hiệu Ω và Θ*) Ta viết $f(n) = \Omega(g(n))$ nếu tồn tại $c > 0$ và n_0 sao cho $f(n) \ge c \cdot g(n)$ với mọi $n \ge n_0$; đây là *cận dưới*.

Ta viết $f(n) = \Theta(g(n))$ nếu đồng thời $f(n) = O(g(n))$ và $f(n) = \Omega(g(n))$; đây là *cận chặt*.

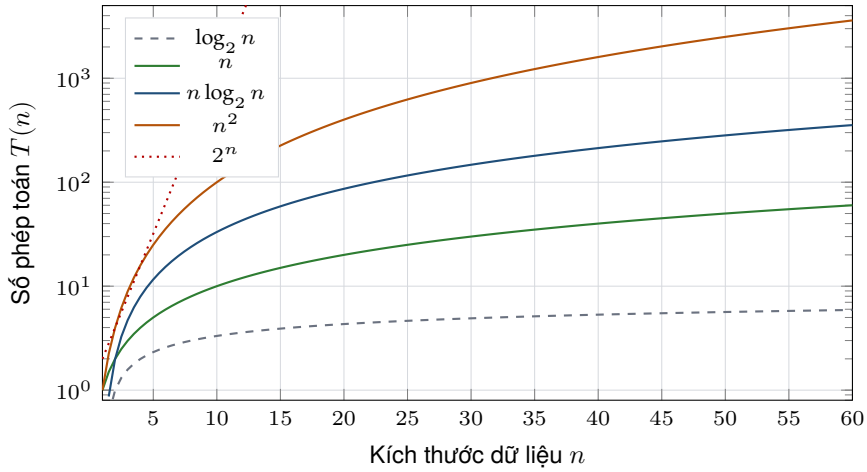
Nhận xét 1.6. [Dùng ký hiệu nào] Trong thực hành, người ta viết O ngay cả khi ý muốn nói Θ, và tài liệu này cũng theo thói quen đó. Chỗ cần phân biệt là khi phát biểu *giới hạn dưới của một bài toán*: câu "mọi thuật toán sắp xếp dựa trên so sánh đều cần $\Omega(n \log n)$ phép so sánh" nói về mọi thuật toán có thể có, nên bắt buộc dùng Ω.

1.3.5 Quy tắc tính độ phức tạp

Bốn quy tắc sau đủ để phân tích hầu hết thuật toán trong giáo trình.

1.3.5.1 Quy tắc hằng số

Một khối lệnh không chứa vòng lặp và không gọi hàm có độ phức tạp $O(1)$, bất kể nó gồm một hay năm mươi câu lệnh, vì số lệnh đó không phụ thuộc n.



Hình 1.4. Tốc độ tăng của các lớp độ phức tạp, trục tung theo thang lô-ga-rit. Đường 2^n vượt khỏi khung hình khi n mới chỉ khoảng 12, còn $\log_2 n$ gần như nằm ngang.

1.3.5.2 Quy tắc cộng

Hai khối lệnh thực hiện tuần tự, khối thứ nhất tốn $O(f)$ và khối thứ hai tốn $O(g)$, thì tổng thời gian là

$$O(f + g) = O(\max(f, g)).$$

Đẳng thức sau đúng vì $\max(f, g) \le f + g \le 2\max(f, g)$, mà hằng số 2 bị bỏ qua. Hệ quả thực dụng: khi một chương trình gồm nhiều giai đoạn, độ phức tạp của cả chương trình bằng độ phức tạp của giai đoạn chậm nhất.

1.3.5.3 Quy tắc nhân

Vòng lặp chạy $O(f)$ lần, mỗi lần thân vòng tốn $O(g)$, thì tổng là $O(f \cdot g)$.

1.3.5.4 Vòng lặp có cận phụ thuộc

Khi cận của vòng trong phụ thuộc biến của vòng ngoài, phải tính *tổng* chứ không nhân trực tiếp.

JULIA

```julia
for i in 1:n
    for j in i+1:n        # vòng trong chạy n - i lần, KHÔNG phải n lần
        # một phép toán O(1)
    end
end
```

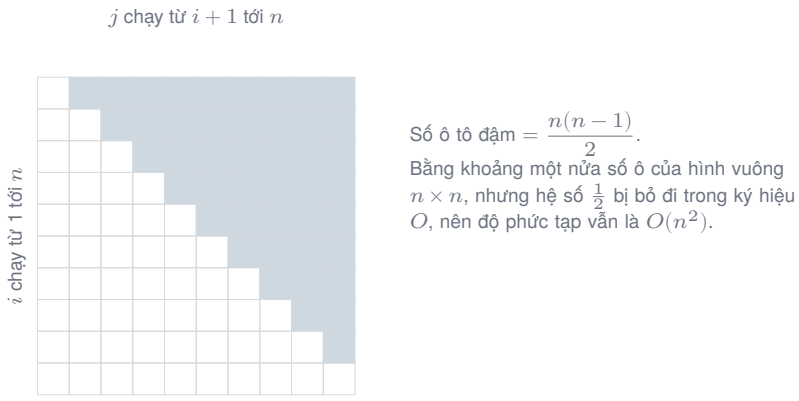
Tổng số lần thực hiện thân vòng là

$$\sum_{i=1}^{n} (n - i) = (n - 1) + (n - 2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

Theo quy tắc bỏ hằng số nhân, độ phức tạp vẫn là $O(n^2)$.

1.3.5.5 Vòng lặp nhân hoặc chia biến đếm

Khi biến đếm được nhân đôi hoặc chia đôi sau mỗi vòng, số vòng lặp là $\log_2 n$.



Hình 1.5. Vòng lặp tam giác. Đây là dạng của cả ba thuật toán sắp xếp cơ bản trong chương 2.

C++

```cpp
int i = 1;
while (i <= n) {
    i = i * 2;        // i nhận 1, 2, 4, 8, ..., nên chạy log2(n) vòng
}
```

Lý do: sau k vòng, $i = 2^k$. Vòng lặp dừng khi $2^k > n$, tức $k > \log_2 n$.

1.3.6 Độ phức tạp của thuật toán đệ quy

Với thuật toán đệ quy, ta lập **hệ thức truy hồi** cho $T(n)$ rồi giải hệ thức đó. Có ba kỹ thuật giải.

1.3.6.1 Kỹ thuật thứ nhất: khai triển liên tiếp

Thay công thức vào chính nó nhiều lần cho tới khi nhìn ra quy luật, rồi kiểm tra lại bằng quy nạp.

Ví dụ 1.4. *Tìm kiếm nhị phân*. Mỗi lần gọi loại bỏ một nửa dãy và thực hiện $O(1)$ việc, cho $T(n) = T(n/2) + 1$ với $T(1) = 1$. Khai triển liên tiếp:

$$T(n) = T(n/2) + 1 = T(n/4) + 2 = T(n/8) + 3 = \cdots = T(n/2^k) + k.$$

Quá trình dừng khi $n/2^k = 1$, tức $k = \log_2 n$, cho $T(n) = T(1) + \log_2 n = O(\log n)$.

Ví dụ 1.5. *Sắp xếp trộn*. Thuật toán chia dãy làm đôi, sắp xếp mỗi nửa rồi trộn lại; bước trộn duyệt một lượt qua n phần tử nên tốn $O(n)$. Ta có $T(n) = 2\,T(n/2) + n$. Khai triển:

$$T(n) = 2\Big(2T(n/4) + \tfrac{n}{2}\Big) + n = 4T(n/4) + 2n = 8T(n/8) + 3n = \cdots = 2^k\,T(n/2^k) + kn.$$

Với $k = \log_2 n$ ta được $T(n) = n\,T(1) + n\log_2 n = O(n \log n)$.

1.3.6.2 Kỹ thuật thứ hai: cây đệ quy

Vẽ cây các lời gọi, ghi chi phí của mỗi nút, rồi cộng theo từng mức. Kỹ thuật này cho lời giải nhanh và trực quan, đặc biệt khi các nhánh không đều nhau. Nó được dùng chi tiết ở chương 4.

1.3.6.3 Kỹ thuật thứ ba: định lý thợ

Định lý 1.2. *(Định lý thợ, dạng đơn giản) Cho* $T(n) = a\,T(n/b) + O(n^d)$ *với* $a \geq 1$, $b > 1$, $d \geq 0$. *Khi đó*

$$T(n) = \begin{cases} O(n^d) & \text{nếu } d > \log_b a, \\ O(n^d \log n) & \text{nếu } d = \log_b a, \\ O(n^{\log_b a}) & \text{nếu } d < \log_b a. \end{cases}$$

Ý nghĩa ba trường hợp: đại lượng $\log_b a$ đo tốc độ *sinh* bài toán con, còn d đo chi phí *gộp* lời giải. Nếu chi phí gộp trội hơn thì tổng chi phí do mức trên cùng quyết định (trường hợp 1); nếu số bài toán con trội hơn thì do mức lá quyết định (trường hợp 3); nếu hai bên cân bằng thì mọi mức đóng góp như nhau và ta nhân thêm số mức, tức $\log n$ (trường hợp 2).

Ví dụ 1.6. *Áp dụng định lý thợ.*

- **Sắp xếp trộn.** $a = 2$, $b = 2$, $d = 1$. Ta có $\log_2 2 = 1 = d$, rơi vào trường hợp thứ hai, cho $T(n) = O(n \log n)$.
- **Tìm kiếm nhị phân.** $a = 1$, $b = 2$, $d = 0$. Ta có $\log_2 1 = 0 = d$, trường hợp thứ hai, cho $T(n) = O(\log n)$.
- **Nhân hai số lớn theo Karatsuba.** $a = 3$, $b = 2$, $d = 1$. Ta có $\log_2 3 \approx 1{,}585 > 1 = d$, trường hợp thứ ba, cho $T(n) = O(n^{1{,}585})$, nhanh hơn cách nhân thông thường $O(n^2)$.

1.3.7 Các lớp độ phức tạp và bảng tra

Bảng 1.11. Các lớp độ phức tạp thường gặp.

Ký hiệu	Tên gọi	Ví dụ thuật toán
$O(1)$	hằng số	Truy cập phần tử mảng; thêm và lấy trên ngăn xếp
$O(\log n)$	lô-ga-rit	Tìm kiếm nhị phân; tìm trên cây cân bằng
$O(n)$	tuyến tính	Tìm giá trị lớn nhất; tìm kiếm tuyến tính; thuật toán Kadane
$O(n \log n)$	tuyến tính lô-ga-rit	Sắp xếp trộn; sắp xếp nhanh; sắp xếp vun đống
$O(n^2)$	bình phương	Sắp xếp nổi bọt; dãy con tăng dài nhất bản cơ bản
$O(n^3)$	lập phương	Nhân hai ma trận; thuật toán Floyd-Warshall
$O(2^n)$	hàm mũ	Liệt kê mọi tập con; quay lui không cắt nhánh
$O(n!)$	giai thừa	Liệt kê mọi hoán vị; bài toán người du lịch vét cạn

Bảng 1.12 là công cụ dùng hằng ngày khi làm bài tập: nhìn giới hạn n trong đề bài, xác định ngay lớp thuật toán cần nhắm tới. Bảng lập theo giả định máy chấm thực hiện khoảng 10^8 phép toán mỗi giây.

Bảng 1.12. Từ giới hạn n suy ra độ phức tạp cần đạt.

Giới hạn n	Độ phức tạp cho phép	Kỹ thuật thường dùng
$n \le 12$	$O(n!)$	Liệt kê hoán vị
$n \le 25$	$O(2^n)$	Liệt kê tập con, quay lui
$n \le 100$	$O(n^4)$	Quy hoạch động trên ma trận
$n \le 500$	$O(n^3)$	Nhân ma trận, Floyd-Warshall
$n \le 5\,000$	$O(n^2)$	Hai vòng lặp lồng nhau
$n \le 10^6$	$O(n \log n)$	Sắp xếp, cấu trúc cây
$n \le 10^8$	$O(n)$	Một lượt duyệt
$n > 10^8$	$O(\log n)$ hoặc $O(1)$	Công thức toán học, tìm kiếm nhị phân

Bảng 1.13. Thời gian ước tính với 10^8 phép toán mỗi giây.

n	n	$n \log_2 n$	n^2	2^n
10	tức thì	tức thì	tức thì	tức thì
10^3	tức thì	tức thì	0,01 giây	không khả thi
10^5	tức thì	0,02 giây	100 giây	không khả thi
10^6	0,01 giây	0,2 giây	3 giờ	không khả thi
10^9	10 giây	5 phút	300 năm	không khả thi

1.3.8 Trường hợp tốt nhất, xấu nhất và trung bình

Với cùng kích thước n, các bộ dữ liệu khác nhau có thể cho số phép toán khác nhau. Vì vậy $T(n)$ không phải một số mà là một khoảng giá trị, và ta xét ba đại lượng.

- **Trường hợp tốt nhất**: số phép toán nhỏ nhất trong mọi bộ dữ liệu kích thước n. Ít có giá trị thực tiễn, vì không ai bảo đảm dữ liệu sẽ thuận lợi.
- **Trường hợp xấu nhất**: số phép toán lớn nhất. Đây là đại lượng quan trọng nhất, vì nó cho một *bảo đảm*: chương trình chắc chắn không chậm hơn thế. Khi nói "độ phức tạp của thuật toán" mà không nói gì thêm, mặc định là trường hợp xấu nhất.
- **Trường hợp trung bình**: kỳ vọng số phép toán, tính theo một phân bố xác suất giả định trên tập dữ liệu vào. Đại lượng này khó tính hơn và phụ thuộc vào phân bố được giả định; nếu phân bố giả định sai thì kết quả không có ý nghĩa.

Ví dụ 1.7. *Ba trường hợp của tìm kiếm tuyến tính*. Duyệt mảng n phần tử để tìm giá trị x.

Tốt nhất: x nằm ở vị trí đầu, tốn 1 phép so sánh.

Xấu nhất: x nằm ở vị trí cuối hoặc không có trong mảng, tốn n phép so sánh.

Trung bình: giả sử x chắc chắn có trong mảng và nằm ở mỗi vị trí với xác suất $1/n$. Số phép so sánh khi x ở vị trí i là i, nên kỳ vọng bằng

$$\sum_{i=1}^{n} \frac{1}{n} \cdot i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

Cả ba trường hợp xấu nhất và trung bình đều thuộc lớp $O(n)$; chúng chỉ khác nhau hằng số 2.

1.3.9 Độ phức tạp không gian

Ngoài thời gian, còn phải đánh giá bộ nhớ. **Độ phức tạp không gian** là lượng bộ nhớ *phụ* mà thuật toán dùng, không kể bộ nhớ chứa dữ liệu vào.

- Thuật toán chỉ dùng thêm một số hữu hạn biến có độ phức tạp không gian $O(1)$, còn gọi là chạy **tại chỗ**.
- Thuật toán cần thêm một mảng cỡ n có độ phức tạp $O(n)$.
- Thuật toán đệ quy tốn bộ nhớ cho ngăn xếp lời gọi, tỉ lệ với *độ sâu* của cây đệ quy chứ không phải với tổng số lời gọi.

Nhận xét 1.7. [Đánh đổi giữa thời gian và bộ nhớ] Rất nhiều kỹ thuật trong giáo trình là đánh đổi hai đại lượng này. Quy hoạch động ở chương 4 dùng thêm một bảng $O(n)$ hoặc $O(n^2)$ để đưa thời gian từ hàm mũ xuống đa thức. Ngược lại, bản cái túi tiết kiệm bộ nhớ ở cùng chương giảm bộ nhớ từ $O(nW)$ xuống $O(W)$ mà giữ nguyên thời gian, đổi lại mất khả năng truy vết lời giải. Khi giới hạn bộ nhớ của đề bài chặt, đây là chỗ đầu tiên cần xem lại.

1.4 Các ví dụ áp dụng

Ba bài toán trong mục này được chọn vì mỗi bài đều có nhiều lời giải với độ phức tạp khác nhau, nên chúng minh hoạ trực tiếp công cụ vừa xây dựng.

1.4.1 Kiểm tra số nguyên tố

> **Bài toán.** Cho số nguyên $n \geq 2$. Kiểm tra n có phải số nguyên tố không, tức có phải chỉ chia hết cho 1 và cho chính nó không.

1.4.1.1 Cách thứ nhất: thử mọi ước

Thuật toán 1.3. *Kiểm tra nguyên tố, thử mọi ước*

1. Nếu $n < 2$ thì trả về *sai*.
2. Với mỗi i từ 2 đến $n - 1$: nếu n chia hết cho i thì trả về *sai* và kết thúc.
3. Trả về *đúng*.

PYTHON

```python
def nguyen_to_cach1(n):
    if n < 2:
        return False
    for i in range(2, n):        # thử mọi ước từ 2 tới n-1
        if n % i == 0:           # tìm được một ước thực sự
            return False         # thoát ngay, không cần thử tiếp
    return True
```

Vòng lặp chạy nhiều nhất $n - 2$ lần, nên độ phức tạp là $O(n)$. Với $n \approx 10^9$, thuật toán cần khoảng một tỉ phép chia, tức khoảng mười giây.

1.4.1.2 Cách thứ hai: chỉ thử tới căn bậc hai

Nhận xét 1.8. [Cơ sở của cải tiến] Nếu $n = p \cdot q$ với $p \le q$ thì $p \le \sqrt{n}$, vì nếu cả hai đều lớn hơn $\sqrt{n}$ thì tích của chúng lớn hơn n. Do đó nếu n có ước thực sự thì nó phải có ít nhất một ước không vượt quá $\sqrt{n}$. Vòng lặp chỉ cần chạy tới $\sqrt{n}$.

Thuật toán 1.4. *Kiểm tra nguyên tố tới căn bậc hai*

1. Nếu $n < 2$ thì trả về *sai*.
2. Với mỗi i từ 2 chừng nào $i \cdot i \le n$: nếu n chia hết cho i thì trả về *sai*.
3. Trả về *đúng*.

JULIA

```julia
function nguyen_to(n)
    n < 2 && return false
    i = 2
    while i * i <= n          # tương đương i <= sqrt(n) nhưng KHÔNG dùng số thực
        n % i == 0 && return false
        i += 1
    end
    return true
end
```

PYTHON

```python
def nguyen_to(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:         # dùng i*i <= n thay cho i <= sqrt(n)
        if n % i == 0:
            return False
        i += 1
    return True
```

C++

```cpp
bool nguyen_to(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {   // long long để i*i không tràn
        if (n % i == 0) return false;
    }
    return true;
}
```

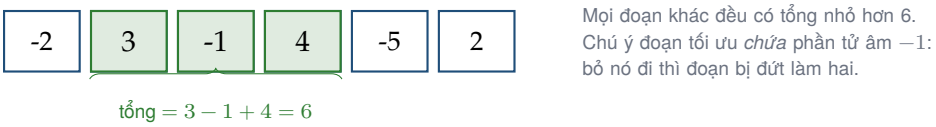
Vì sao viết `i * i <= n` chứ không `i <= sqrt(n)`. Hàm `sqrt` trả về số thực, và số thực có sai số làm tròn như đã nêu ở mục 1.1.5. Với n là một số chính phương lớn, `sqrt(n)` có thể cho một giá trị nhỏ hơn căn đúng một lượng cực nhỏ, khiến vòng lặp bỏ sót đúng ước $\sqrt{n}$ và kết luận sai. Điều kiện `i * i <= n` chỉ dùng số nguyên nên không có rủi ro đó. Đổi lại, trong C++ phải khai báo `i` kiểu `long long`, vì với n cỡ 10^{18} thì $i \cdot i$ vượt khoảng của `int`.

Độ phức tạp là $O(\sqrt{n})$. Với $n \approx 10^9$, số phép chia giảm từ một tỉ xuống khoảng 31 600, tức nhanh hơn khoảng ba mươi nghìn lần.

1.4.2 Đoạn con liên tiếp có tổng lớn nhất

> **Bài toán.** Cho dãy n số nguyên có thể âm. Tìm đoạn gồm các phần tử *liên tiếp* có tổng lớn nhất. Đoạn phải có ít nhất một phần tử.

Bài toán này đáng nghiên cứu kỹ vì cùng một đề có ba lời giải với ba độ phức tạp khác nhau, và mỗi lần cải tiến đều xuất phát từ một nhận xét cụ thể chứ không phải từ mẹo vặt.



Hình 1.6. Đoạn con liên tiếp có tổng lớn nhất của dãy $-2, 3, -1, 4, -5, 2$.

1.4.2.1 Cách thứ nhất: duyệt mọi đoạn, cộng lại từ đầu

Thuật toán 1.5. *Đoạn con tổng lớn nhất, ba vòng lặp*

1. Khởi tạo `best` bằng a_1.
2. Với mỗi chỉ số đầu i từ 1 tới n:
   (a) Với mỗi chỉ số cuối j từ i tới n:
      i. Tính tổng s của đoạn $a_i..a_j$ bằng một vòng lặp thứ ba.
      ii. Nếu $s >$ `best` thì cập nhật `best` $\leftarrow s$.
3. Trả về `best`.

JULIA

```julia
function cach1(a)
    n = length(a)
    best = a[1]                # khởi tạo bằng a[1], KHÔNG bằng 0
    for i in 1:n               # chỉ số đầu đoạn
        for j in i:n           # chỉ số cuối đoạn
            s = 0
            for k in i:j       # vòng lặp thứ ba: cộng lại từ đầu đoạn
                s = s + a[k]
            end
            if s > best
                best = s
            end
        end
    end
    return best
end
```

PYTHON

```python
def cach1(a):
    n = len(a)
    best = a[0]
    for i in range(n):                  # chỉ số đầu đoạn
        for j in range(i, n):           # chỉ số cuối đoạn
            s = 0
            for k in range(i, j + 1):   # vòng lặp thứ ba
                s = s + a[k]
            if s > best:
                best = s
    return best
```

C++

```cpp
long long cach1(const vector<int> &a) {
    int n = a.size();
    long long best = a[0];              // long long vì tổng có thể rất lớn
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++) {
            long long s = 0;
            for (int k = i; k <= j; k++) s += a[k];   // vòng lặp thứ ba
            if (s > best) best = s;
        }
    return best;
}
```

Vì sao khởi tạo `best` bằng a_1 chứ không bằng 0. Nếu khởi tạo bằng 0, thuật toán trả về 0 khi mọi phần tử đều âm, nhưng 0 không phải tổng của đoạn nào cả (đoạn phải có ít nhất một phần tử). Đáp số đúng khi đó là phần tử lớn nhất, tức phần tử âm gần 0 nhất. Đây là lỗi rất hay gặp và chỉ lộ ra khi bộ dữ liệu thử có toàn số âm.

Phân tích. Ba vòng lặp lồng nhau, mỗi vòng chạy tới n lần, cho $O(n^3)$. Chính xác hơn, số lần chạy thân vòng trong cùng là $\sum_i \sum_{j \geq i} (j - i + 1) = \Theta(n^3)$.

1.4.2.2 Cách thứ hai: cộng dồn

Nhận xét 1.9. [Cơ sở của cải tiến] Khi cố định chỉ số đầu i và tăng dần chỉ số cuối j, tổng của đoạn $a_i..a_{j+1}$ bằng tổng của đoạn $a_i..a_j$ cộng thêm a_{j+1}. Vậy không cần cộng lại từ đầu; vòng lặp thứ ba bị loại bỏ hoàn toàn.

JULIA

```julia
function cach2(a)
    n = length(a)
    best = a[1]
    for i in 1:n
        s = 0                   # tổng của đoạn bắt đầu từ i, khởi tạo lại mỗi lần
        for j in i:n
            s = s + a[j]        # CỘNG DỒN thay vì tính lại từ đầu
            if s > best
                best = s
            end
        end
```

```
    end
    return best
end
```

PYTHON

```python
def cach2(a):
    n = len(a)
    best = a[0]
    for i in range(n):
        s = 0                           # đặt lại cho mỗi chỉ số đầu i
        for j in range(i, n):
            s = s + a[j]                # cộng dồn
            if s > best:
                best = s
    return best
```

C++

```cpp
long long cach2(const vector<int> &a) {
    int n = a.size();
    long long best = a[0];
    for (int i = 0; i < n; i++) {
        long long s = 0;
        for (int j = i; j < n; j++) {
            s += a[j];                  // cộng dồn
            if (s > best) best = s;
        }
    }
    return best;
}
```

Độ phức tạp giảm còn $O(n^2)$.

1.4.2.3 Cách thứ ba: thuật toán Kadane

Nhận xét 1.10. [Cơ sở của cải tiến] Gọi cur là tổng lớn nhất của đoạn kết thúc *đúng tại* vị trí i. Đoạn đó chỉ có hai khả năng: hoặc nó chỉ gồm mình a_i, hoặc nó là đoạn tối ưu kết thúc tại $i-1$ nối thêm a_i. Vậy

$$\texttt{cur}_i = \max(a_i,\ \texttt{cur}_{i-1} + a_i).$$

Đáp số là giá trị lớn nhất trong tất cả các $\texttt{cur}_i$.

Thuật toán 1.6. *Kadane*

1. Khởi tạo best $\leftarrow a_1$ và cur $\leftarrow a_1$.
2. Với mỗi i từ 2 tới n:
   (a) cur $\leftarrow \max(a_i,\ \texttt{cur} + a_i)$: quyết định bắt đầu đoạn mới tại i hay nối tiếp đoạn cũ.
   (b) best $\leftarrow \max(\texttt{best},\ \texttt{cur})$: ghi nhận kỷ lục.
3. Trả về best.

GIẢ MÃ

```
KADANE(a, n)
1  best ← a[1];  cur ← a[1]
2  for i ← 2 to n
3      cur ← max(a[i], cur + a[i])    // bắt đầu lại, hay nối tiếp?
4      best ← max(best, cur)          // ghi nhận kỷ lục
5  return best
```

Giải thích từng bước.

Dòng	Ý nghĩa và lý do có dòng này
1	Khởi tạo cả hai biến bằng a_1. Đoạn kết thúc tại vị trí 1 chỉ có một khả năng là chính a_1, nên `cur` $= a_1$; và khi mới xét một phần tử thì kỷ lục cũng bằng a_1.
3	Dòng quyết định. Nếu `cur` trước đó âm thì a_i một mình đã lớn hơn `cur` $+ a_i$, nên thuật toán bỏ hẳn đoạn cũ và bắt đầu đoạn mới tại i. Nếu `cur` dương thì nối tiếp có lợi hơn.
4	Ghi nhận riêng. Không được trả về `cur` ở cuối, vì `cur` chỉ là tổng của đoạn kết thúc tại phần tử *cuối cùng*, chưa chắc là đoạn tốt nhất. Cần một biến riêng giữ giá trị lớn nhất từng thấy.

Định lý 1.3. (*Tính đúng đắn của thuật toán Kadane*) *Sau khi xử lý phần tử thứ i, biến* `best` *bằng tổng lớn nhất trong mọi đoạn con liên tiếp không rỗng của $a[1..i]$.*

Chứng minh. **Bất biến.** Trước lần lặp với chỉ số i: `cur` bằng tổng lớn nhất của các đoạn kết thúc đúng tại $i-1$, và `best` bằng tổng lớn nhất của mọi đoạn nằm trong $a[1..i-1]$.

Khởi tạo. Với $i = 2$, đoạn duy nhất kết thúc tại vị trí 1 là chính a_1, và đoạn duy nhất nằm trong $a[1..1]$ cũng là a_1. Dòng 1 gán đúng cả hai giá trị đó.

Duy trì. Mọi đoạn kết thúc tại i hoặc chỉ gồm a_i, hoặc có dạng "một đoạn kết thúc tại $i-1$, nối thêm a_i". Tổng lớn nhất trong nhóm thứ hai bằng $\texttt{cur}_{i-1} + a_i$ theo giả thiết quy nạp. Vậy dòng 3 tính đúng $\texttt{cur}_i$. Tiếp theo, mọi đoạn nằm trong $a[1..i]$ hoặc nằm hẳn trong $a[1..i-1]$, hoặc kết thúc tại i; giá trị lớn nhất của hai nhóm này lần lượt là `best` cũ và $\texttt{cur}_i$, nên dòng 4 tính đúng `best` mới.

Kết thúc. Vòng lặp dừng khi $i = n + 1$. Áp bất biến: `best` bằng tổng lớn nhất của mọi đoạn nằm trong $a[1..n]$, tức đáp số cần tìm. ∎

JULIA

```julia
function kadane(a)
    best = a[1]
    cur  = a[1]
    for i in 2:length(a)
        cur  = max(a[i], cur + a[i])   # bắt đầu lại, hay nối tiếp?
        best = max(best, cur)          # ghi nhận kỷ lục
    end
    return best
end
```

PYTHON

```python
def kadane(a):
    best = a[0]
    cur  = a[0]
    for i in range(1, len(a)):
        cur  = max(a[i], cur + a[i])   # bắt đầu lại, hay nối tiếp?
        best = max(best, cur)          # ghi nhận kỷ lục
    return best
```

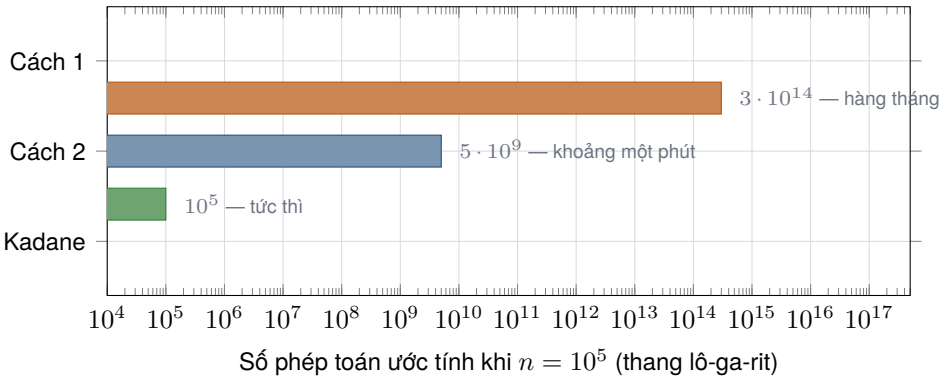
C++

```cpp
long long kadane(const vector<int> &a) {
    long long best = a[0], cur = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        // ép sang long long trước khi cộng để tránh tràn ở tổng
        cur  = max((long long)a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

1.4.2.4 So sánh ba cách

Bảng 1.14. Ba lời giải cho bài toán đoạn con có tổng lớn nhất.

Cách	Nhận xét dẫn tới cải tiến	Độ phức tạp	Số phép toán khi $n = 10^5$
Cách 1	—	$O(n^3)$	$\approx 3 \cdot 10^{14}$
Cách 2	Tổng đoạn mở rộng tính được từ tổng đoạn cũ	$O(n^2)$	$\approx 5 \cdot 10^9$
Kadane	Đoạn kết thúc tại i chỉ có hai khả năng	$O(n)$	10^5



Hình 1.7. Số phép toán của ba lời giải cho bài toán đoạn con có tổng lớn nhất, với $n = 10^5$.

Bài 1.2. Cài đặt cả ba cách bằng ba ngôn ngữ, theo các bước sau.

1. Kiểm thử với dãy $(-2, 3, -1, 4, -5, 2)$; cả ba phải trả về 6.
2. Kiểm thử với dãy toàn số âm, chẳng hạn $(-5, -2, -9)$; cả ba phải trả về -2. Nếu một bản trả về 0 thì bản đó khởi tạo `best` sai.

3. Viết chương trình kiểm thử tự động: sinh 3000 dãy ngẫu nhiên độ dài từ 1 đến 9 với giá trị từ -9 đến 9, so sánh kết quả của ba cách với nhau và với một chương trình vét cạn viết riêng.
4. Đo thời gian của cách 2 và Kadane với $n = 10^4, 10^5, 10^6$. Cách 1 chỉ đo được với $n \le 500$.
5. Sửa Kadane để trả về thêm *chỉ số đầu và chỉ số cuối* của đoạn tối ưu, không chỉ tổng. Gợi ý: ghi lại vị trí bắt đầu mỗi khi dòng 3 chọn nhánh "bắt đầu lại".

1.4.3 Dãy Fibonacci

Bài toán. Dãy Fibonacci định nghĩa bằng $F_1 = F_2 = 1$ và $F_n = F_{n-1} + F_{n-2}$ với $n \ge 3$. Cho n, tính F_n.

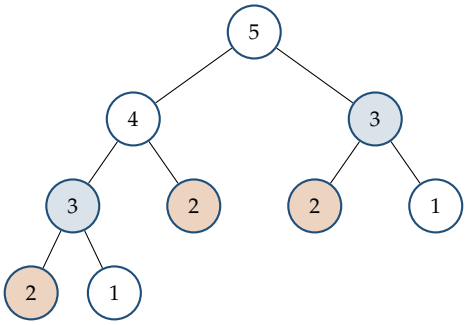
1.4.3.1 Cách thứ nhất: đệ quy thuần

Định nghĩa dịch thẳng sang chương trình đệ quy.

PYTHON

```python
def fib(n):
    if n <= 2:
        return 1                # hai số đầu tiên đều bằng 1
    return fib(n - 1) + fib(n - 2)
```

Chương trình này đúng nhưng chậm khủng khiếp. Hình 1.8 chỉ ra nguyên nhân: cùng một giá trị bị tính đi tính lại rất nhiều lần.



Nút tô nhạt: `fib(3)` bị tính **hai** lần.

Nút tô cam: `fib(2)` bị tính **ba** lần.

Với n lớn, số lần tính lặp lại tăng theo hàm mũ: tổng số lời gọi bằng $2F_n - 1$, tức cỡ $O(1{,}618^n)$.

Hình 1.8. Cây các lời gọi khi tính `fib(5)` bằng đệ quy thuần.

Số đo thực tế: tính F_{20} cần 13 529 lời gọi, tính F_{30} cần 1 664 079 lời gọi, và tính F_{50} cần khoảng $2{,}5 \times 10^{10}$ lời gọi, tức khoảng một phút chỉ để cộng dồn 50 con số.

1.4.3.2 Cách thứ hai: ghi nhớ kết quả

Nhận xét 1.11. [Cơ sở của cải tiến] Chương trình chậm không phải vì đệ quy mà vì *tính lại*. Nếu mỗi giá trị F_k được tính đúng một lần rồi ghi vào một bảng, các lần cần sau chỉ việc tra bảng.

JULIA

```julia
bo_nho = Dict{Int, Int}()          # bảng ghi nhớ: khoá là n, giá trị là F(n)

function fib_memo(n)
```

```
    n <= 2 && return 1
    haskey(bo_nho, n) && return bo_nho[n]      # đã tính rồi thì tra bảng
    bo_nho[n] = fib_memo(n-1) + fib_memo(n-2)  # tính, ghi vào bảng, trả về
end
```

PYTHON

```python
bo_nho = {}                        # từ điển làm bảng ghi nhớ

def fib_memo(n):
    if n <= 2:
        return 1
    if n in bo_nho:
        return bo_nho[n]           # đã tính rồi thì tra bảng
    bo_nho[n] = fib_memo(n-1) + fib_memo(n-2)
    return bo_nho[n]
```

C++

```cpp
map<int, long long> bo_nho;        // bảng ghi nhớ

long long fib_memo(int n) {
    if (n <= 2) return 1;
    if (bo_nho.count(n)) return bo_nho[n];     // đã tính rồi
    return bo_nho[n] = fib_memo(n-1) + fib_memo(n-2);
}
```

1.4.3.3 Cách thứ ba: bỏ đệ quy

Tính lần lượt từ nhỏ đến lớn và chỉ giữ hai giá trị gần nhất.

JULIA

```
function fib_lap(n)
    n <= 2 && return 1
    a, b = 1, 1              # a = F(k-1), b = F(k)
    for _ in 3:n
        a, b = b, a + b      # dịch cửa sổ: F(k-1) mới là F(k) cũ
    end
    return b
end
```

PYTHON

```python
def fib_lap(n):
    if n <= 2:
        return 1
    a, b = 1, 1              # a = F(k-1), b = F(k)
    for _ in range(3, n + 1):
        a, b = b, a + b      # dịch cửa sổ sang phải một bước
    return b
```

C++

```cpp
long long fib_lap(int n) {
    if (n <= 2) return 1;
    long long a = 1, b = 1;
    for (int i = 3; i <= n; i++) {
        long long c = a + b; // C++ không có gán đồng thời nên cần biến c
        a = b;
        b = c;
    }
    return b;
}
```

Bảng 1.15. Ba cách tính F_n.

Cách	Thời gian	Bộ nhớ phụ	Ghi chú
Đệ quy thuần	$O(1{,}618^n)$	$O(n)$ ngăn xếp	Không dùng được với $n > 40$
Ghi nhớ	$O(n)$	$O(n)$ bảng	Giữ được dạng đệ quy dễ đọc
Vòng lặp	$O(n)$	$O(1)$	Nhanh nhất và tốn ít nhớ nhất

Với $n = 50$, hai cách sau cho kết quả $F_{50} = 12\,586\,269\,025$ ngay lập tức.

Nhận xét 1.12. Ý tưởng ghi nhớ kết quả của các bài toán con là hạt nhân của quy hoạch động, sẽ được trình bày đầy đủ ở chương 4. Điều đáng chú ý là cải tiến này không đổi thuật toán: công thức truy hồi vẫn nguyên như cũ, chỉ thêm một bảng tra. Đó là một trong những cách cải tiến rẻ nhất trong toàn bộ giáo trình.

1.4.4 Đo thời gian chạy thực tế

Sau khi phân tích lý thuyết, nên tự kiểm chứng bằng cách đo.

JULIA

```julia
# @time in ra thời gian, số lần cấp phát bộ nhớ và lượng bộ nhớ đã dùng.
# Chạy hai lần: lần đầu bao gồm cả thời gian biên dịch tức thời.
@time fib_lap(1_000_000)
@time fib_lap(1_000_000)
```

PYTHON

```python
import time

t0 = time.perf_counter()          # perf_counter chính xác hơn time.time
fib_lap(100000)
t1 = time.perf_counter()
print(f"Thoi gian: {t1 - t0:.4f} giay")
```

C++

```cpp
#include <chrono>
using namespace std::chrono;

auto bat_dau = high_resolution_clock::now();
fib_lap(100000);
```

```cpp
auto ket_thuc = high_resolution_clock::now();
cout << duration<double, milli>(ket_thuc - bat_dau).count() << " ms" << endl;
```

Đo thời gian cho đúng. Ba nguyên tắc: chạy mỗi phép đo nhiều lần rồi lấy giá trị nhỏ nhất chứ không lấy trung bình, vì các lần chậm là do hệ điều hành chen ngang; với Julia phải bỏ lần chạy đầu tiên vì nó bao gồm thời gian biên dịch; và luôn ghi rõ tuỳ chọn biên dịch, vì `-O2` và không có `-O2` cho kết quả chênh nhau vài lần.

1.5 Ứng dụng: thuật toán láng giềng gần nhất

1.5.1 Bài toán phân lớp

Bài toán phân lớp. Cho một tập **mẫu huấn luyện**, mỗi mẫu gồm một véc-tơ **đặc trưng** $x \in \mathbb{R}^d$ và một **nhãn** y thuộc một tập hữu hạn các lớp. Cho một mẫu mới chỉ có véc-tơ đặc trưng. Hãy dự đoán nhãn của nó.

Đây là bài toán trung tâm của học máy có giám sát. Chẩn đoán bệnh từ các chỉ số xét nghiệm, phân loại thư rác từ các từ trong thư, nhận dạng chữ viết tay từ các điểm ảnh — tất cả đều là bài toán phân lớp.

1.5.2 Ý tưởng

Thuật toán phân lớp đơn giản nhất dựa trên một giả thiết trực quan: *hai mẫu có đặc trưng gần giống nhau thì nhiều khả năng thuộc cùng một lớp*. Từ đó suy ra cách làm: để dự đoán nhãn cho mẫu mới, tìm mẫu huấn luyện nào giống nó nhất rồi trả về nhãn của mẫu đó.

Để máy so sánh được mức độ giống nhau, ta phải lượng hoá nó thành số. Cách thông dụng nhất là coi mỗi mẫu là một điểm trong không gian d chiều và dùng **khoảng cách Euclid**:

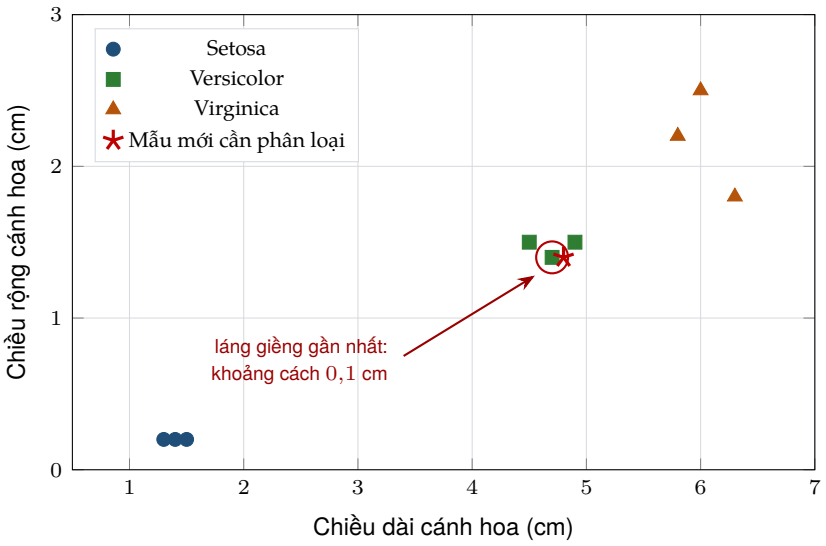
$$d(u, v) = \sqrt{\sum_{k=1}^{d} (u_k - v_k)^2}.$$

Với $d = 2$ đây chính là công thức Pythagore quen thuộc. Hai mẫu càng giống nhau thì khoảng cách càng nhỏ.

Ví dụ 1.8. *Bộ dữ liệu hoa diên vĩ rút gọn.* Mỗi bông hoa được mô tả bằng hai đặc trưng: chiều dài và chiều rộng cánh hoa, đo bằng xăng-ti-mét. Nhãn là loài hoa.

Mẫu	Chiều dài	Chiều rộng	Loài
1	1,4	0,2	Setosa
2	1,3	0,2	Setosa
3	1,5	0,2	Setosa
4	4,7	1,4	Versicolor
5	4,5	1,5	Versicolor
6	4,9	1,5	Versicolor
7	6,0	2,5	Virginica
8	5,8	2,2	Virginica
9	6,3	1,8	Virginica

Mẫu mới cần phân loại có đặc trưng $(4{,}8,\ 1{,}4)$.



Hình 1.9. Bộ dữ liệu hoa diên vĩ rút gọn. Mẫu mới được gán nhãn của điểm gần nó nhất.

1.5.3 Thuật toán

Thuật toán 1.7. *Láng giềng gần nhất*

Dữ liệu vào: tập huấn luyện D gồm n cặp (đặc trưng, nhãn); mẫu mới x. Dữ liệu ra: nhãn dự đoán cho x.

1. Khởi tạo `kc_nhỏ_nhất` $\leftarrow +\infty$ và `kết_quả` $\leftarrow$ rỗng.
2. Với mỗi mẫu (x_i, y_i) trong D:
   (a) Tính $d \leftarrow$ khoảng cách Euclid từ x tới x_i.
   (b) Nếu $d <$ `kc_nhỏ_nhất` thì đặt `kc_nhỏ_nhất` $\leftarrow d$ và `kết_quả` $\leftarrow y_i$.
3. Trả về `kết_quả`.

GIẢ MÃ

```
LANG-GIENG-GAN-NHAT(mau_moi, D)
1  kc_nho_nhat ← +∞
2  ket_qua ← rỗng
3  for mỗi mẫu trong D
4       d ← khoảng cách từ mau_moi tới mẫu đó
5       if d < kc_nho_nhat
6            kc_nho_nhat ← d
7            ket_qua ← nhãn của mẫu đó
8  return ket_qua
```

Giải thích từng bước.

Dòng	Ý nghĩa và lý do có dòng này
1	Khởi tạo bằng vô cùng lớn là bắt buộc. Nếu khởi tạo bằng 0 thì điều kiện ở dòng 5 không bao giờ đúng, vì mọi khoảng cách đều không âm, và hàm sẽ trả về kết quả rỗng.
3–4	Vòng lặp duyệt *toàn bộ* tập huấn luyện. Không có cách nào bỏ bớt, vì mẫu gần nhất có thể nằm ở bất kỳ đâu trong tập.
5–7	Ba dòng này là khuôn mẫu "tìm giá trị nhỏ nhất kèm thông tin đi theo". Điểm cần chú ý là phải cập nhật *cả hai* biến cùng lúc: quên dòng 7 thì hàm biết khoảng cách nhỏ nhất nhưng không biết nhãn tương ứng.

So sánh giả mã này với thuật toán `TIM-MAX` ở mục 1.2.5, ta thấy chúng có cùng cấu trúc. Khác biệt chỉ ở chỗ tìm giá trị nhỏ nhất thay vì lớn nhất, và ở chỗ trả về nhãn đi kèm chứ không trả về chính giá trị.

1.5.4 Cài đặt

PYTHON

```python
import math

def khoang_cach(u, v):
    """Khoảng cách Euclid giữa hai véc-tơ cùng số chiều."""
    tong = 0.0
    for k in range(len(u)):
        tong += (u[k] - v[k]) ** 2      # bình phương chênh lệch từng chiều
    return math.sqrt(tong)

def lang_gieng_gan_nhat(mau_moi, D):
    """D là danh sách các cặp (dac_trung, nhan)."""
    kc_nho_nhat = float("inf")          # vô cùng lớn, KHÔNG khởi tạo bằng 0
    ket_qua = None
    for dac_trung, nhan in D:
        d = khoang_cach(mau_moi, dac_trung)
        if d < kc_nho_nhat:
            kc_nho_nhat = d             # cập nhật kỷ lục
            ket_qua = nhan              # và nhãn đi kèm
    return ket_qua, kc_nho_nhat
```

JULIA

```julia
# Julia cho phép viết công thức gọn nhờ phép toán theo từng phần tử:
# u .- v trừ từng cặp phần tử, .^2 bình phương từng phần tử.
khoang_cach(u, v) = sqrt(sum((u .- v) .^ 2))

function lang_gieng_gan_nhat(mau_moi, D)
    kc_nho_nhat = Inf                   # Inf là vô cùng trong Julia
    ket_qua = nothing
    for (dac_trung, nhan) in D
        d = khoang_cach(mau_moi, dac_trung)
        if d < kc_nho_nhat
            kc_nho_nhat = d
            ket_qua = nhan
        end
    end
    return ket_qua, kc_nho_nhat
end
```

```
end
```

C++

```cpp
struct Mau {
    vector<double> x;    // véc-tơ đặc trưng
    string y;            // nhãn
};

double khoang_cach(const vector<double> &u, const vector<double> &v) {
    double tong = 0;
    for (size_t k = 0; k < u.size(); k++)
        tong += (u[k] - v[k]) * (u[k] - v[k]);
    return sqrt(tong);
}

pair<string, double> lang_gieng_gan_nhat(const vector<double> &mau_moi,
                                         const vector<Mau> &D) {
    double kc_nho_nhat = 1e18;          // "vô cùng" trong thực hành
    string ket_qua = "";
    for (const Mau &m : D) {            // tham chiếu để không sao chép mỗi mẫu
        double d = khoang_cach(mau_moi, m.x);
        if (d < kc_nho_nhat) {
            kc_nho_nhat = d;
            ket_qua = m.y;
        }
    }
    return {ket_qua, kc_nho_nhat};
}
```

Ví dụ 1.9. *Kết quả trên bộ dữ liệu hoa diên vĩ.* Mẫu mới $(4{,}8,\ 1{,}4)$ có khoảng cách tới mẫu 4 là

$$\sqrt{(4{,}8-4{,}7)^2+(1{,}4-1{,}4)^2}=0{,}1,$$

nhỏ hơn khoảng cách tới mọi mẫu khác. Vì mẫu 4 mang nhãn Versicolor, thuật toán dự đoán mẫu mới là Versicolor.

1.5.5 Đánh giá độ phức tạp

Gọi n là số mẫu huấn luyện và d là số đặc trưng.

- Mỗi lần tính khoảng cách duyệt d chiều nên tốn $O(d)$.
- Vòng lặp chạy n lần.
- Tổng cộng: $O(n \cdot d)$ cho *mỗi* mẫu cần phân loại.
- Bộ nhớ phụ: $O(1)$, ngoài bản thân tập huấn luyện.

Nhận xét 1.13. [Đặc điểm của thuật toán này trong học máy] Thuật toán láng giềng gần nhất không có giai đoạn *huấn luyện*: nó chỉ lưu lại toàn bộ dữ liệu. Toàn bộ chi phí dồn vào giai đoạn *dự đoán*. Điều đó trái ngược với hầu hết mô hình học máy khác, vốn tốn nhiều thời gian huấn luyện nhưng dự đoán rất nhanh.

Hệ quả thực tiễn: với $n = 10^6$ mẫu và $d = 100$ đặc trưng, mỗi lần dự đoán tốn 10^8 phép toán, tức khoảng một giây. Một hệ thống phục vụ hàng nghìn yêu cầu mỗi giây không dùng được cách

này. Việc tăng tốc phải dựa vào cấu trúc dữ liệu chuyên dụng — cây k-d hoặc cây bao trùm — chứ không thể dựa vào máy tính mạnh hơn. Đây là ví dụ đầu tiên trong giáo trình cho thấy lựa chọn cấu trúc dữ liệu quyết định một hệ thống có chạy được ở quy mô thực tế hay không.

Thuật toán không "hiểu" gì cả. Chương trình vừa viết không hề biết hoa là gì. Nó chỉ tính khoảng cách giữa các bộ số và trả về nhãn của bộ số gần nhất. Toàn bộ "tri thức" nằm trong tập dữ liệu huấn luyện, không nằm trong chương trình. Đặc điểm này đúng với hầu hết thuật toán học máy, kể cả những mô hình phức tạp hơn nhiều.

Bài 1.3. Cài đặt thuật toán láng giềng gần nhất bằng cả ba ngôn ngữ, theo các bước sau.

1. Nạp bộ dữ liệu chín mẫu ở ví dụ trên vào chương trình.
2. Viết hàm `khoang_cach` và kiểm thử riêng: khoảng cách giữa $(1{,}4,\ 0{,}2)$ và $(4{,}7,\ 1{,}4)$ phải bằng $\sqrt{3{,}3^2 + 1{,}2^2} \approx 3{,}511$.
3. Chạy với mẫu mới $(4{,}8,\ 1{,}4)$; kết quả phải là Versicolor với khoảng cách $0{,}1$.
4. In ra *toàn bộ* chín khoảng cách đã sắp xếp tăng dần, để thấy rõ mẫu 4 đứng đầu.
5. Thử với mẫu mới $(3{,}0,\ 0{,}8)$, nằm giữa hai cụm. Nhận xét về độ tin cậy của dự đoán trong trường hợp này, và giải thích vì sao chương 2 phải mở rộng thuật toán thành k láng giềng.
6. Đo thời gian dự đoán khi tập huấn luyện có 10^3, 10^4, 10^5 mẫu sinh ngẫu nhiên với $d = 10$. Xác nhận thời gian tăng tuyến tính theo n.

Tóm tắt. Chương này đặt ba nền tảng. Về lập trình: biến là ô nhớ có tên, kiểu dữ liệu quy định miền giá trị và số byte, ba câu lệnh điều khiển và mảng đủ để viết mọi thuật toán trong giáo trình. Về khái niệm: thuật toán là dãy hữu hạn bước xác định, và tính đúng đắn của nó được chứng minh bằng bất biến vòng lặp chứ không bằng chạy thử. Về đánh giá: ký hiệu O bỏ qua hằng số và số hạng bậc thấp để so sánh các thuật toán theo tốc độ tăng, và bảng tra ở mục 1.3.7 cho phép từ giới hạn của đề bài suy ra ngay lớp thuật toán cần nhắm tới. Bài toán đoạn con có tổng lớn nhất minh hoạ trọn vẹn ý nghĩa thực tiễn của điều đó: ba lời giải cho cùng một đề, chênh nhau từ hàng tháng xuống còn tức thì, mà không cần máy tính nhanh hơn.

1.6 Bài tập chương 1

Câu hỏi ôn tập

1. Nêu ba cách chuyển mã nguồn thành lệnh máy và cho biết ngôn ngữ nào trong tài liệu dùng cách nào.
2. Vì sao tính chất truy cập ngẫu nhiên của bộ nhớ chính là cơ sở để nói truy cập phần tử mảng tốn $O(1)$?
3. Phân biệt lệnh gán trong lập trình với dấu bằng trong toán học. Giải thích ý nghĩa của câu lệnh `x = x + 1`.
4. Kiểu dữ liệu quy định những gì? Vì sao C++ bắt buộc khai báo kiểu còn Python thì không?

5. Hiện tượng tràn số nguyên là gì? Vì sao `long long c = a + a;` vẫn tràn khi `a` là `int`?
6. Vì sao không được so sánh hai số thực bằng dấu ==? Nêu cách so sánh đúng.
7. Nêu ba khác biệt của phép chia giữa ba ngôn ngữ.
8. Tính chất short-circuit của phép `và` là gì? Cho một ví dụ trong đó thiếu tính chất này thì chương trình sai.
9. Vì sao thứ tự các nhánh trong một chuỗi `if--else if` lại quan trọng?
10. Nêu sáu đặc trưng của thuật toán, mỗi đặc trưng kèm một phản ví dụ.
11. Phân biệt thuật toán, giải thuật và chương trình.
12. Nêu bốn cách mô tả thuật toán và ưu nhược điểm của từng cách.
13. Bất biến vòng lặp là gì? Nêu ba điều phải kiểm tra khi dùng nó để chứng minh tính đúng đắn.
14. Hàm biến thiên dùng để chứng minh điều gì? Nêu hàm biến thiên của thuật toán Euclid.
15. Phát biểu định nghĩa hình thức của ký hiệu O. Vì sao trong định nghĩa lại có cả hằng số c lẫn ngưỡng n_0?
16. Phân biệt O, Ω và Θ. Khi nào bắt buộc phải dùng Ω?
17. Phát biểu định lý thợ và giải thích ý nghĩa của ba trường hợp.
18. Phân biệt trường hợp tốt nhất, xấu nhất và trung bình. Vì sao trường hợp xấu nhất là đại lượng quan trọng nhất?
19. Độ phức tạp không gian của thuật toán đệ quy tỉ lệ với đại lượng nào?
20. Trong thuật toán Kadane, vì sao cần hai biến `cur` và `best` riêng biệt thay vì một biến?

Bài tập tính toán

21. Chứng minh trực tiếp từ định nghĩa: $5n^3 + 2n^2 + 100 = O(n^3)$. Chỉ rõ c và n_0.
22. Chứng minh $n^2 \neq O(n)$ bằng phản chứng.
23. Sắp xếp các hàm sau theo tốc độ tăng: n, $\log n$, $n \log n$, n^2, 2^n, $n!$, $\sqrt{n}$, $n^{1,5}$, $\log \log n$.
24. Tính $T(n)$ chính xác (không dùng ký hiệu O) cho đoạn mã có ba vòng lặp lồng nhau với cận i từ 1 tới n, j từ i tới n, k từ j tới n.
25. Giải các hệ thức truy hồi: $T(n) = T(n-1) + n$; $T(n) = 2T(n/2) + n^2$; $T(n) = 4T(n/2) + n$; $T(n) = 3T(n/3) + n$.
26. Lần vết thuật toán Euclid với $(1071, 462)$ và với $(17, 5)$.
27. Lần vết thuật toán Kadane trên dãy $(4, -3, 5, -2, -1, 2, 6, -9)$, ghi giá trị `cur` và `best` sau mỗi bước.
28. Viết bất biến vòng lặp và chứng minh tính đúng đắn cho thuật toán tính tổng các phần tử của mảng.
29. Đề bài cho $n \leq 2000$ và giới hạn thời gian 1 giây. Tra bảng 1.12 để xác định độ phức tạp cao nhất được chấp nhận, và giải thích.
30. Chứng minh rằng nếu n có ước thực sự thì nó có ước không vượt quá $\sqrt{n}$.

Bài tập lập trình mức cơ bản

31. Viết chương trình đọc n số nguyên rồi in ra tổng, trung bình cộng, giá trị nhỏ nhất và lớn nhất. Cài bằng cả ba ngôn ngữ.

32. Viết chương trình kiểm tra một số có phải số chính phương không, không dùng hàm `sqrt`.

33. Viết chương trình đếm số chữ số của một số nguyên dương và tính tổng các chữ số đó.

34. Viết chương trình đảo ngược một số nguyên: từ 12345 cho ra 54321.

35. Viết chương trình in ra mọi số nguyên tố nhỏ hơn hoặc bằng n bằng cách kiểm tra từng số. Sau đó cài sàng Eratosthenes và so sánh thời gian với $n = 10^6$.

36. Viết chương trình tính $n!$ và cho biết giá trị n lớn nhất mà kiểu `long long` còn biểu diễn được kết quả.

37. Viết chương trình đếm số phần tử chẵn, số phần tử lẻ và số phần tử âm trong một mảng.

38. Viết chương trình kiểm tra một mảng đã sắp xếp tăng dần hay chưa, độ phức tạp $O(n)$.

Bài tập lập trình mức vận dụng

39. Cài đặt cả ba cách giải bài toán đoạn con có tổng lớn nhất theo bài thực hành 1.2, kèm chương trình kiểm thử tự động.

40. Sửa thuật toán Kadane để trả về thêm chỉ số đầu và chỉ số cuối của đoạn.

41. Cho mảng n số nguyên, tìm đoạn con liên tiếp có *tích* lớn nhất. Chú ý xử lý số 0 và số âm.

42. Cho mảng n số nguyên và số k, tìm tổng lớn nhất của một đoạn con có đúng k phần tử liên tiếp, độ phức tạp $O(n)$. Kỹ thuật này gọi là cửa sổ trượt.

43. Cài đặt hàm tính F_n bằng cả ba cách trong mục 1.4, thêm biến đếm số lời gọi, và xác nhận các số 13 529 với $n = 20$ và 1 664 079 với $n = 30$.

44. Viết chương trình đo và vẽ đồ thị thời gian chạy của một thuật toán $O(n^2)$ khi n tăng, rồi kiểm chứng bằng thực nghiệm rằng tăng n gấp đôi làm thời gian tăng khoảng bốn lần.

45. Cài đặt hàm tính $a^b \bmod m$ với b tới 10^{18}, độ phức tạp $O(\log b)$.

Bài tập phần trí tuệ nhân tạo

46. Cài đặt thuật toán láng giềng gần nhất theo bài thực hành 1.3.

47. Mở rộng chương trình để đọc tập huấn luyện từ tệp văn bản với số đặc trưng tuỳ ý, không cố định bằng 2.

48. Thay khoảng cách Euclid bằng khoảng cách Manhattan $\sum_{k} |u_k - v_k|$ và so sánh kết quả phân loại trên cùng tập dữ liệu.

49. Viết chương trình đánh giá độ chính xác bằng phương pháp bỏ một mẫu: với mỗi mẫu trong tập, tạm bỏ nó ra, phân loại nó bằng các mẫu còn lại, rồi đếm tỉ lệ dự đoán đúng.

50. Đo thời gian phân loại 1000 mẫu mới khi tập huấn luyện có 10^3, 10^4, 10^5 mẫu và $d = 20$. Lập bảng và xác nhận thời gian tỉ lệ thuận với n.

51. Tìm hiểu và trình bày ngắn gọn ý tưởng của cây k-d, cấu trúc dữ liệu đưa thời gian tìm láng giềng gần nhất từ $O(nd)$ xuống $O(d \log n)$ trung bình khi d nhỏ.